INFORME DE PROYECTO

Diseño e implementación de una CNC láser controlada mediante una placa NUCLEO-F446RE

Nicolas Patricelli[1]

Federico Barrios Retta[2]

[1]Facultad de Ingeniería, Universidad Nacional de Cuyo, Argentina
[2]Facultad de Ingeniería, Universidad Nacional de Cuyo, Argentina

Resumen

Se presenta el diseño y la implementación de una CNC láser basada en una placa NUCLEO-F446RE. El firmware interpreta instrucciones G-code enviadas desde LaserGRBL, coordina el movimiento CoreXY y el eje Z mediante motores paso a paso y genera el mando PWM del cabezal. Se describen la estructura mecánica, las interfaces eléctricas, la organización modular, la referenciación y el tratamiento de límites. Las pruebas funcionales permitieron completar el homing de los tres ejes, observar la detención ante finales de carrera y obtener un grabado sobre MDF de 5 mm de espesor nominal. La validación disponible es cualitativa: no se determinaron valores de exactitud o repetibilidad y no se considera validada la capacidad de corte. Se identifican las limitaciones de la implementación y las mejoras de protección, acondicionamiento y diagnóstico previstas.

Índice

1. Introducción y objetivos **3**
1.1. Objetivo general . . . 3
1.2. Objetivos específicos . . . 3

2. Descripción general del sistema **3**

3. Sistema mecánico, hardware e interfaces eléctricas **4**
3.1. Armadura de la CNC . . . 4
3.2. Microcontrolador seleccionado . . . 4
3.3. Motores seleccionados . . . 5
3.4. Láser seleccionado . . . 6
3.5. Drivers seleccionados . . . 7
3.5.1. TB6600 – Accionamiento de los motores . . . 7
3.5.2. P-DA-01 – Interfaz del láser . . . 8
3.6. Conexiones eléctricas . . . 9

4. Arquitectura del firmware **12**
4.1. Organización modular . . . 12
4.2. Inicialización y programa principal . . . 14
4.3. Coordinación de los módulos . . . 15
4.4. Ejecución mediante interrupciones . . . 15
4.5. Representación de los estados de operación . . . 15

5. Comunicación e interpretación de G-code **17**
5.1. Generación y envío de instrucciones mediante LaserGRBL . . . 17
5.2. Comunicación serie y tratamiento de mensajes . . . 17
5.3. Interpretación de las instrucciones . . . 17
5.4. Comandos reconocidos y estado modal . . . 18
5.5. Anticipación de segmentos mediante lookahead . . . 19
5.6. Alcance de la compatibilidad con LaserGRBL . . . 19

6. Generación y coordinación del movimiento **20**
6.1. Cinemática y elección del mecanismo CoreXY . . . 20
6.2. Conversión de desplazamientos en pulsos . . . 21
6.3. Interpolación y sincronización de los motores . . . 22
6.4. Temporización y perfil de velocidad . . . 22
6.5. Ejecución del barrido de grabado . . . 23
6.6. Movimiento independiente del eje Z . . . 23
6.7. Resolución y exactitud del posicionamiento . . . 24

7. Referenciación, límites y control del láser **24**
7.1. Referenciación de los ejes . . . 24
7.2. Finales de carrera y bloqueo por alarma . . . 25
7.3. Límites del espacio de trabajo . . . 25
7.4. Accionamiento y modulación del láser . . . 25

8. Ensayos y resultados **26**
8.1. Alcance de las pruebas . . . 26
8.2. Referenciación y respuesta a límites . . . 26
8.3. Prueba de grabado . . . 27
8.4. Síntesis de los resultados . . . 27

9. Limitaciones y conclusiones **27**
9.1. Limitaciones y alcance de la validación . . . 27
9.2. Mejoras previstas . . . 28
9.3. Conclusiones . . . 29

1. Introducción y objetivos

El grabado mediante láser requiere que el desplazamiento relativo entre la herramienta y la pieza sea coordinado con el accionamiento del láser. Para ejecutar una trayectoria definida, deben ser interpretadas las instrucciones de trabajo y generadas las señales que gobiernan los actuadores de la máquina.

En el marco de la asignatura *Microcontroladores y Electrónica de Potencia*, se desarrolló el control de una CNC destinada al grabado y corte láser. El proyecto permite integrar los contenidos relacionados con la programación de sistemas embebidos, la comunicación serie, la utilización de sensores y el accionamiento de motores paso a paso.

1.1. Objetivo general

Diseñar e implementar un sistema de control basado en un microcontrolador para ejecutar trayectorias de grabado mediante láser a partir de instrucciones enviadas desde una computadora.

1.2. Objetivos específicos

Se plantean como objetivos específicos la interpretación de las instrucciones de trabajo, la coordinación de los desplazamientos en el plano de grabado, el ajuste de la distancia relativa entre el láser y la pieza, y la regulación del accionamiento del láser durante la ejecución de las trayectorias.

Asimismo, se busca establecer una referencia de posición mediante sensores de fin de carrera y verificar el funcionamiento del sistema mediante ensayos de movimiento y grabado.

2. Descripción general del sistema

La máquina desarrollada consiste en un sistema de control numérico computarizado (CNC) destinado al grabado y al corte mediante láser. Durante estos procesos, el cabezal es desplazado respecto de la pieza siguiendo una trayectoria definida por instrucciones G-code, mientras que el accionamiento del láser es coordinado con el movimiento.

El posicionamiento del cabezal en el plano X–Y se realiza mediante un mecanismo de cinemática CoreXY, accionado por dos motores paso a paso. Se incorpora, además, un eje Z que permite desplazar el cabezal para ajustar la distancia entre el módulo láser y la superficie de trabajo. El eje Z es accionado de manera independiente de los desplazamientos coordinados en el plano X–Y.

Las trayectorias de trabajo son definidas mediante un archivo G-code generado en una computadora. Las instrucciones son transmitidas al microcontrolador STM32 de una placa NUCLEO-F446RE, donde son interpretadas para generar las señales de paso, dirección y habilitación de los drivers de los motores. Asimismo, se genera una señal de modulación por ancho de pulso (PWM) destinada a comandar el accionamiento del módulo láser.

El sistema incorpora finales de carrera para establecer una referencia de posición mediante un procedimiento de referenciación o *homing*. A partir de dicha referencia, los desplazamientos son determinados mediante las consignas y los pasos comandados a los motores. El posicionamiento se realiza en lazo abierto, sin medición continua de la posición del cabezal.

La Figura 1 resume los bloques funcionales y distingue las consignas de movimiento, el mando del láser y las entradas de referencia.

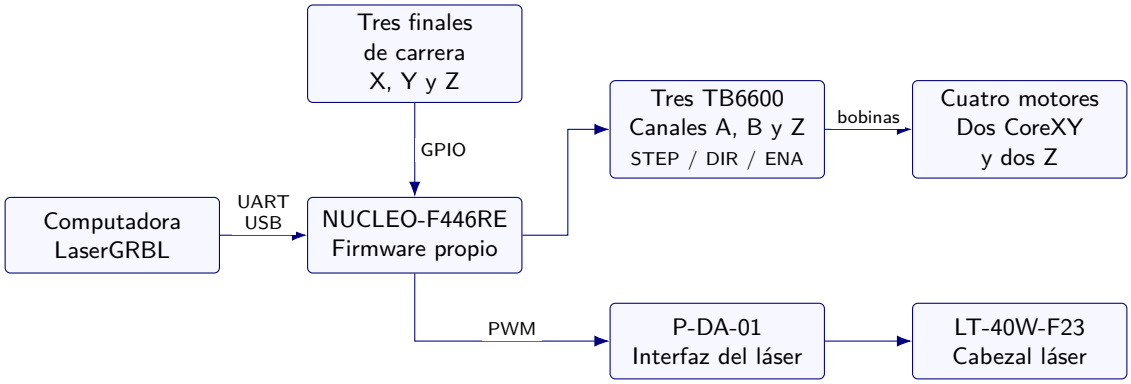


Figura 1. Bloques funcionales de la CNC. Las conexiones de potencia y las referencias eléctricas se detallan en la Figura 8.

3. Sistema mecánico, hardware e interfaces eléctricas

3.1. Armadura de la CNC

La armadura se construyó a partir del diseño *Core XY*, publicado por Javierarnedo en Thingiverse [1]. Se conservaron los perfiles y la configuración del bastidor. No se dispone de un relevamiento dimensional del montaje; los límites configurados en el firmware no se utilizan como medidas del bastidor ni como recorridos útiles comprobados.

Se utilizaron perfiles ranurados de aluminio, dispuestos en dos marcos rectangulares unidos mediante cuatro montantes. Las uniones atornilladas y las escuadras permiten fijar los soportes de guiado y transmisión. La alineación del bastidor y la ausencia de holguras en las uniones condicionan la geometría del recorrido.

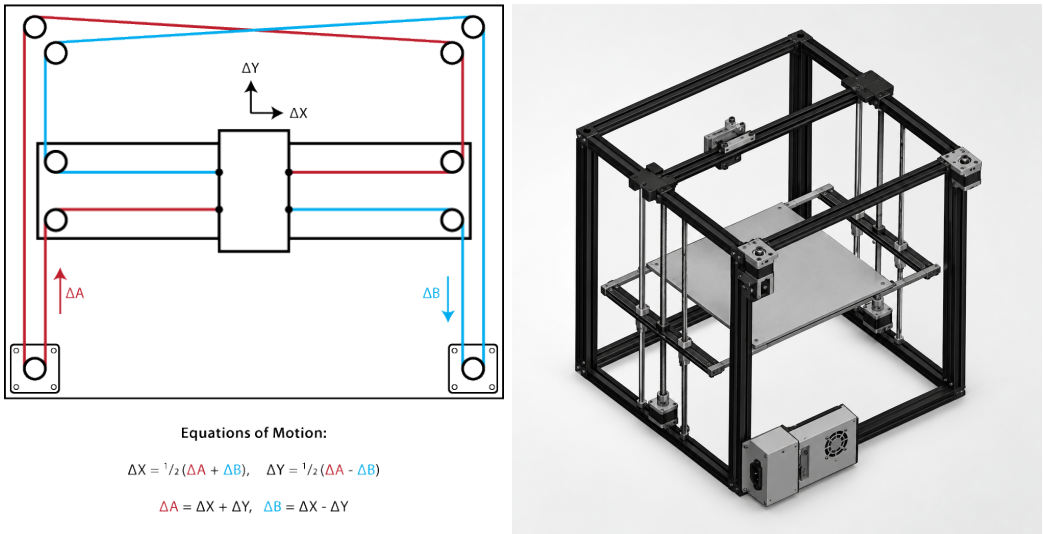


Figura 2. Diseño mecánico de referencia y recorrido de correas CoreXY [1]. Las imágenes ilustran la arquitectura adoptada; no constituyen fotografías del prototipo láser ni representan su mecanismo Z definitivo.

3.2. Microcontrolador seleccionado

El sistema de control fue implementado mediante una placa de desarrollo NUCLEO-F446RE, que incorpora un microcontrolador STM32F446RET6 de la familia STM32F4. Este dispositivo dispone de un núcleo Arm Cortex-M4 de 32 bits, con unidad de punto flotante de precisión simple, 512 KB de memoria Flash y 128 KB de memoria SRAM [9].

La unidad de punto flotante proporciona soporte para las operaciones aritméticas empleadas en el procesamiento de coordenadas y en los cálculos de movimiento. Aunque el microcontrolador admite una frecuencia máxima de 180 MHz, en la configuración del proyecto se utiliza un reloj de sistema de 84 MHz.

Para la temporización del movimiento se utilizan los temporizadores TIM2 y TIM4, asociados a la generación de pasos de los motores del mecanismo CoreXY y del eje Z, respectivamente. El canal 1 del temporizador TIM3 se utiliza para generar la señal PWM que comanda el módulo láser.

La comunicación con la computadora se realiza mediante USART2, configurada a 115200 bit/s. La interfaz de puerto serie virtual de la placa permite establecer esta comunicación a través de la conexión USB del ST-LINK. Asimismo, este dispositivo integrado permite programar y depurar el microcontrolador durante el desarrollo.

Las señales de paso, dirección y habilitación de los drivers se generan mediante pines GPIO. También se utilizan entradas digitales para recibir las señales de los finales de carrera. Los conectores de expansión de la placa permiten acceder a los pines necesarios para estas interfaces.

La disponibilidad de temporizadores, comunicación serie, salidas PWM y pines de entrada y salida permite concentrar en un mismo microcontrolador la interpretación de instrucciones, la coordinación del movimiento y el comando de los actuadores.

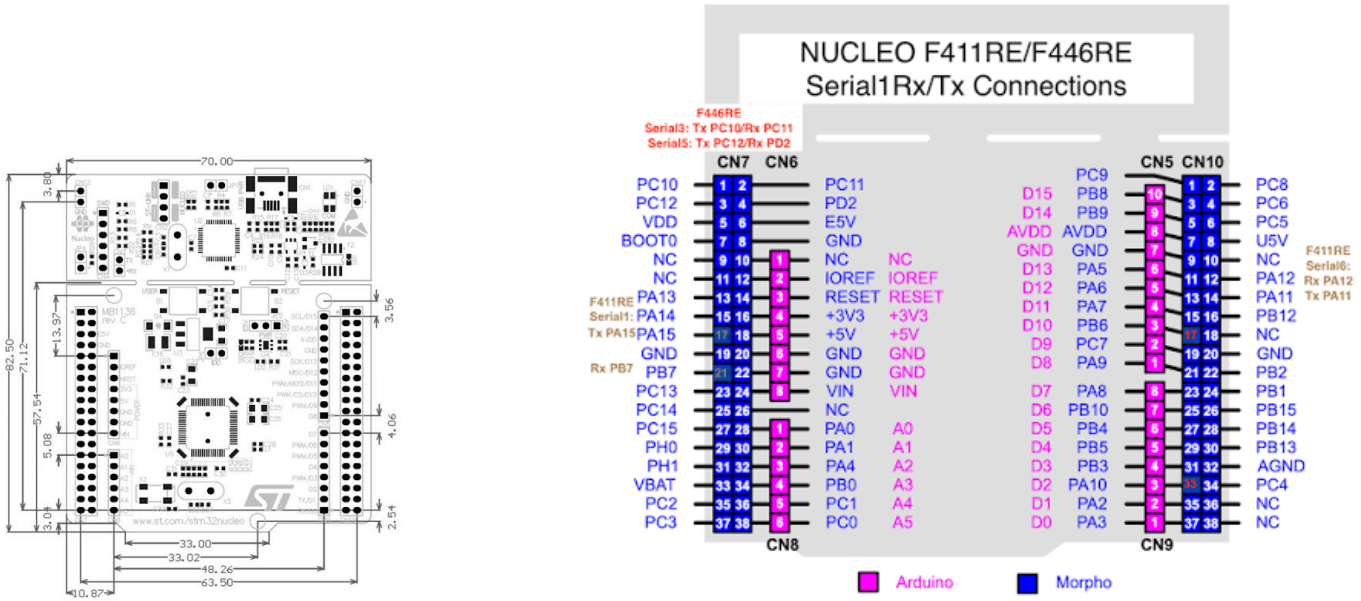


Figura 3. Referencia mecánica de la placa NUCLEO-64 y asignación de conectores. Para el proyecto se utiliza la variante F446RE; las señales empleadas se reúnen en la Tabla 4.

3.3. Motores seleccionados

Para el accionamiento de la máquina se utilizaron cuatro motores paso a paso bipolares de tamaño de brida NEMA 17, marca Usongshine, modelo `17HS3401`.

Dos motores accionan el mecanismo CoreXY, correspondientes a los canales A y B. Los otros dos se utilizan para el desplazamiento del eje Z y se conectan en paralelo a un mismo driver.

Cada motor dispone de dos bobinas, cuyo accionamiento requiere invertir el sentido de la corriente para establecer la secuencia de excitación. Esta función es realizada por los módulos TB6600, mientras que el microcontrolador genera las señales de paso, dirección y habilitación.

La denominación NEMA 17 identifica el tamaño de la brida de montaje y no determina por sí sola la corriente nominal, el ángulo de paso ni el par disponible. No se dispone de la ficha específica de corriente y par del ejemplar instalado; estos datos no se deducen del tamaño de brida.

En el firmware se utiliza un ángulo de paso de $1{,}8°$, equivalente a 200 pasos completos por revolución, y una subdivisión de $1/16$ de paso. La conversión a pulsos por revolución y por milímetro se desarrolla en la Sección 6. Estos parámetros deben coincidir con el ángulo nominal del motor y con el ajuste de micropaso seleccionado en los drivers.

El ajuste eléctrico y su relación con la conexión de los motores se describen en la Subsección 3.5. No se dispone de un registro de las posiciones DIP utilizadas.

Tabla 1. Identificación y características de los motores utilizados.

Parámetro	Identificación o valor
Marca	Usongshine
Modelo	17HS3401
Tamaño de brida	NEMA 17
Tipo de accionamiento	Bipolar, dos fases
Cantidad instalada	Cuatro motores
Lote relevado	202331120
Ángulo utilizado por el firmware	$1{,}8^\circ$
Corriente y par nominales	Sin ficha específica disponible

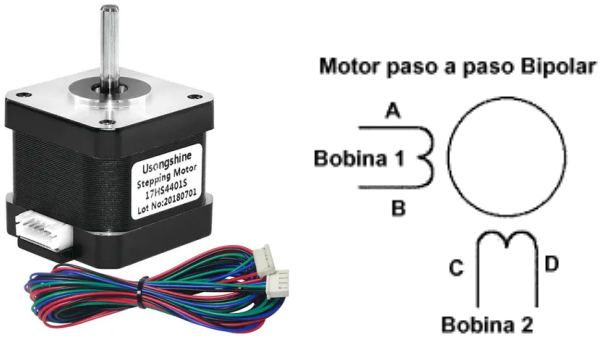


Figura 4. Esquema funcional de las dos bobinas de un motor bipolar. Las letras identifican extremos de bobina; no establecen el orden de terminales ni los colores del motor instalado.

3.4. Láser seleccionado

Como herramienta de trabajo se utiliza un módulo láser LASER TREE LT-40W-F23, destinado a operaciones de grabado y corte. El conjunto integra el emisor láser, la óptica de enfoque, la electrónica de accionamiento y un sistema de refrigeración mediante disipador y ventilador.

Según la documentación del fabricante [2], su potencia óptica es de 5 ± 1 W y su longitud de onda es de 450 ± 10 nm, correspondiente a la región azul del espectro visible. La denominación LT-40W-F23 identifica el modelo y no debe interpretarse como una potencia óptica de 40 W.

Las principales características declaradas por el fabricante se presentan en la Tabla 2.

Tabla 2. Características declaradas del módulo láser LASER TREE LT-40W-F23.

Parámetro	**Especificación**
Potencia óptica	5 ± 1 W
Longitud de onda	450 ± 10 nm
Tensión de alimentación	12 V CC
Corriente de entrada	1,5–2 A
Distancia de enfoque declarada	23 mm
Frecuencia de modulación	Hasta 5 kHz
Nivel de mando indicado	0/5–12 V
Temperatura de operación	0–60 °C
Conector	XH2.54 de tres terminales
Masa del módulo	230 g

La interfaz del cabezal utiliza tres conductores: alimentación positiva, referencia GND y entrada PWM. Esta disposición permite mantener la alimentación del módulo y modificar su accionamiento mediante la señal de control, sin utilizar la salida del microcontrolador para suministrar la corriente del emisor.

El cabezal se encuentra montado sobre el mecanismo de desplazamiento del eje Z. Este movimiento permite ajustar su posición respecto de la pieza para establecer la condición de enfoque. Para el grabado, el fabricante indica que el foco

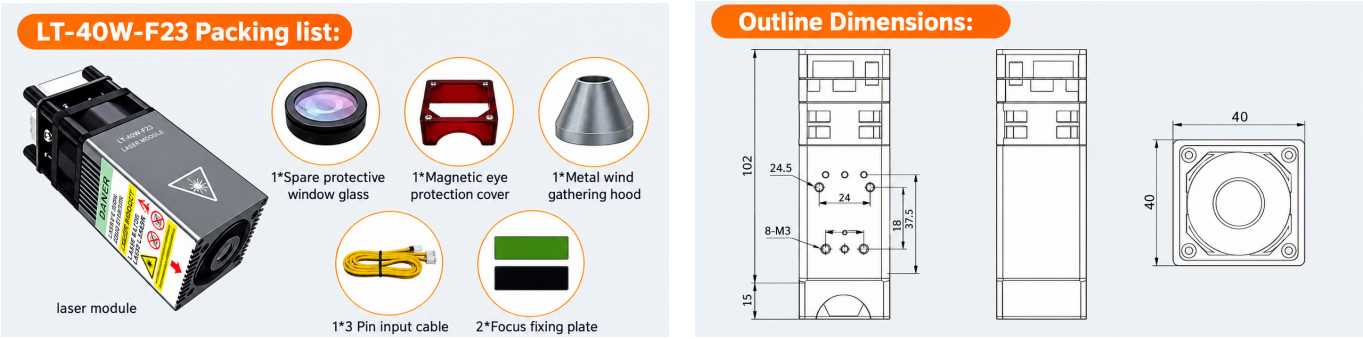


Figura 5. Cabezal LT-40W-F23, accesorios y dimensiones de referencia del fabricante. Estas imágenes de producto no son fotografías del montaje experimental.

debe situarse sobre la superficie del material. La referencia mecánica utilizada para realizar ese ajuste debe respetar las indicaciones de montaje del módulo.

El accionamiento se establece mediante PWM; los modos y la relación entre consigna y salida se desarrollan en la Sección 7. La potencia óptica indicada en la tabla es un dato del fabricante, no una medición del proyecto.

La posibilidad de grabar o cortar una pieza depende del material, su espesor, el enfoque, la velocidad de avance y la consigna de accionamiento. Por ello, las prestaciones de la máquina deben establecerse mediante ensayos, sin deducirse únicamente de la potencia nominal del cabezal.

3.5. Drivers seleccionados

3.5.1. TB6600 – Accionamiento de los motores

Para el accionamiento de los motores paso a paso se utilizaron módulos driver comercializados como TB6600. Su función consiste en recibir las señales de comando generadas por el microcontrolador y establecer la excitación de los bobinados necesaria para producir el movimiento.

Mediante estos módulos se distingue la generación de las consignas de movimiento del suministro de energía a los motores. El microcontrolador determina la cantidad de pulsos, su frecuencia y el sentido de desplazamiento, mientras que el driver gobierna las corrientes de los bobinados a partir de su alimentación de potencia.

Interfaz de comando. El movimiento se comanda mediante las entradas PUL y DIR. Cada pulso válido recibido en PUL produce un avance en la secuencia de excitación, cuya magnitud depende de la subdivisión de pasos seleccionada. La entrada DIR establece el sentido de avance de dicha secuencia. Además, se dispone de una entrada ENA para controlar la habilitación del accionamiento.

Las entradas de comando se presentan mediante pares de terminales PUL+/PUL-, DIR+/DIR- y ENA+/ENA-. Según las especificaciones disponibles, estas entradas son optoacopladas y están destinadas a señales de 5 V, con una corriente de señal comprendida entre 8 y 15 mA. Por lo tanto, su conexionado debe considerar tanto la tensión aplicada como la corriente requerida por la interfaz de entrada.

Configuración de corriente y micropasos. El módulo permite seleccionar la corriente de salida mediante interruptores DIP. Este ajuste establece el nivel de corriente utilizado para la excitación de los bobinados y debe determinarse a partir de las características del motor conectado. La corriente máxima admitida por el driver no constituye, por sí sola, el valor que debe utilizarse en el proyecto.

También puede seleccionarse la subdivisión de pasos. En el funcionamiento por micropasos, las corrientes de los bobinados son modificadas para generar posiciones intermedias entre los pasos completos del motor. Esto permite incrementar la cantidad de pulsos requeridos por vuelta y reducir el incremento angular nominal asociado a cada pulso.

La posición de los interruptores debe corresponder a la subdivisión utilizada por el firmware. En Z, la corriente del driver se reparte entre los dos motores conectados en paralelo; por ello, el ajuste debe considerar las dos ramas y no solamente un motor. La conversión de micropasos a desplazamiento se desarrolla en la Sección 6.

Características eléctricas. En la Tabla 3 se resumen las especificaciones recopiladas para el módulo. La alimentación del montaje se documenta en el esquema de conexiones. Los demás valores proceden de la información comercial recopilada; no se dispone de una ficha que identifique la versión del módulo ni de un registro de sus ajustes DIP.

Parámetro	Valor declarado
Tensión de alimentación de potencia	9–42 V CC
Intervalo de corriente de salida	0,7–4,0 A
Niveles de corriente seleccionables	8
Frecuencia máxima de pulsos de entrada	Hasta 20 kHz
Tensión de las señales de entrada	5 V
Corriente de las señales de entrada	8–15 mA
Pulsos por vuelta para un motor de $1{,}8^\circ$	200–6400
Aislamiento de las entradas	Optoacoplado
Protecciones declaradas	Sobrecorriente y sobretemperatura

Tabla 3. Especificaciones recopiladas del módulo TB6600 según la información comercial aportada. No se identifica la revisión del módulo ni si la corriente declarada es pico o eficaz.

La frecuencia de PUL gobierna la velocidad de la secuencia de excitación. El límite de 20 kHz es un dato declarado para el módulo, no una velocidad experimental de la CNC; el par disponible también depende de la alimentación, la corriente y la velocidad.

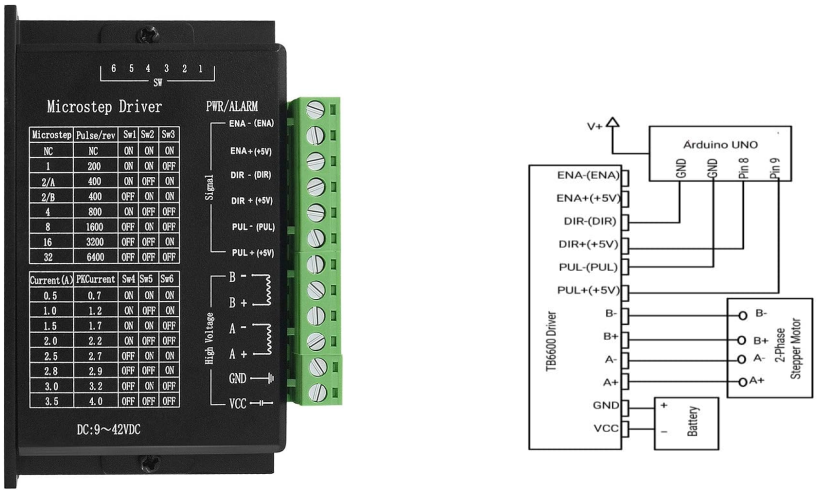


Figura 6. Aspecto ilustrativo de un módulo TB6600. El conexionado de la CNC se presenta en la Figura 8.

3.5.2. P-DA-01 – Interfaz del láser

La conexión entre el sistema de control, la alimentación y el cabezal láser se realiza mediante una placa adaptadora P-DA-01. Su función es facilitar la interconexión de estos elementos mediante conectores de alimentación y de señal TTL/PWM.

Debe distinguirse esta placa del driver propio del cabezal, encargado del accionamiento del diodo láser. La P-DA-01 constituye una interfaz de conexión y distribución de alimentación [2]; la documentación disponible no permite atribuirle funciones de regulación de corriente ni de aislamiento galvánico.

La placa dispone de distintas entradas para adaptar el conexionado de la unidad de control y de salidas de tres terminales hacia el módulo láser. Estas últimas comprenden la alimentación positiva, la referencia GND y la señal de mando PWM. También incorpora una entrada de alimentación externa de corriente continua y un selector EXT/INT, mediante el cual se selecciona la alimentación externa o la procedente del conexionado de entrada. Dos indicadores luminosos permiten comprobar la presencia de alimentación y de la señal de control.

En el firmware del proyecto, la señal de mando del láser es generada por el canal 1 del temporizador TIM3 y se encuentra asignada al pin PC6 del microcontrolador. Su frecuencia es de 1 kHz y su ciclo de trabajo se modifica para establecer la

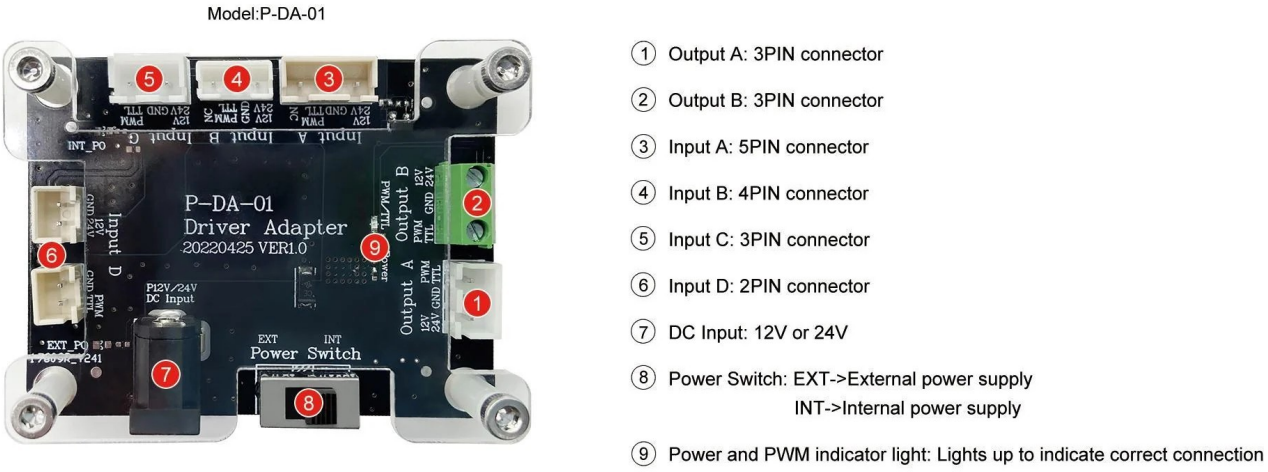


Figura 7. Interfaz P-DA-01 y descripción ilustrativa de sus conexiones. La cantidad de terminales puede variar entre revisiones; el orden del montaje no se deduce únicamente de la imagen

consigna de accionamiento del cabezal. La alimentación del láser es suministrada por la fuente de la máquina, mientras que la salida del microcontrolador proporciona únicamente la señal de mando.

3.6. Conexiones eléctricas

Las interconexiones entre la placa de desarrollo, los drivers de los motores, los finales de carrera y el módulo de accionamiento del láser se representan en la Figura 8. El esquema fue elaborado en KiCad e identifica los terminales de los módulos, las redes de alimentación y las señales de control utilizadas por el firmware.

Distribución de la alimentación. Se dispone de una fuente con entrada de $220\,\mathrm{V}$ de corriente alterna y salidas de $24\,\mathrm{V}$, $12\,\mathrm{V}$ y $5\,\mathrm{V}$ de corriente continua. La salida de 12 V también se representa como alimentación del cabezal, en correspondencia con la tabla específica del LT-40W-F23. Aunque los diagramas genéricos del manual mencionan 12/24 V, esa indicación no se interpreta como autorización para alimentar este modelo con 24 V. La alimentación de potencia de los drivers TB6600 se realiza mediante la salida de $12\,\mathrm{V}$, con los tres módulos conectados en paralelo. La placa NUCLEO-F446RE se alimenta desde la computadora mediante la conexión USB del ST-LINK, que también proporciona el puerto serie virtual.

En el esquema se distinguen las alimentaciones de potencia, las tensiones utilizadas por las interfaces de control y sus correspondientes retornos. Las uniones entre referencias eléctricas se representan según el cableado del sistema.

Interfaz con los drivers de los motores. El accionamiento de los motores es comandado mediante señales de paso, dirección y habilitación. Las señales de paso y dirección son independientes para los canales A, B y Z, mientras que la habilitación es compartida por los tres drivers.

El cableado documentado en el firmware corresponde a una conexión de ánodo común: los terminales `PUL+`, `DIR+` y `ENA+` se conectan a $3{,}3\,\mathrm{V}$, y los terminales negativos reciben las señales de los GPIO. La correspondencia entre señales y pines se presenta en la Tabla 4. El uso de 3,3 V difiere del nivel comercial declarado de 5 V para los TB6600. Sin el circuito de entrada de la versión instalada ni mediciones de corriente y umbral, no se afirma que el conexionado proporcione los márgenes eléctricos requeridos.

Conexión de los motores. Cada motor bipolar dispone de dos bobinas. Los terminales `A+` y `A−` del driver se conectan a los extremos de una bobina, y los terminales `B+` y `B−`, a los extremos de la otra.

Los motores CoreXY se identifican como M1 (A) y M2 (B) en el esquema. Cada uno dispone de un driver; estas letras deben distinguirse de A+/A- y B+/B-, que identifican las bobinas. Para el eje Z, el firmware documenta dos motores conectados en paralelo a un mismo driver. El esquema muestra las ramificaciones de cada salida hacia M3 y M4, correspondientes a los dos motores de Z.

Tabla 4. Asignación de señales de control y entradas de límite en la NUCLEO-F446RE.

Señal	GPIO	Destino o procedencia
STEP_A	PC0	PUL− del driver A
DIR_A	PC1	DIR− del driver A
STEP_B	PC2	PUL− del driver B
DIR_B	PC3	DIR− del driver B
STEP_Z	PC4	PUL− del driver Z
DIR_Z	PC5	DIR− del driver Z
MOTOR_EN	PC7	ENA− de los tres drivers
LIMIT_X	PB0	Señal del final de carrera X
LIMIT_Y	PB1	Señal del final de carrera Y
LIMIT_Z	PB2	Señal del final de carrera Z
LASER_PWM	PC6	Entrada de control del módulo láser

Interfaz con los finales de carrera. Se utilizan tres señales de límite, correspondientes a X, Y y Z, conectadas a PB0, PB1 y PB2, respectivamente. Los módulos de final de carrera se representan mediante sus terminales de alimentación, masa y señal, conservando la identificación del conector utilizado.

El firmware configura estas entradas con resistencias internas de polarización a masa e interpreta un nivel bajo como la activación del límite. Por lo tanto, debe comprobarse que la salida de los módulos presente el nivel de reposo y de activación esperado por el programa. El esquema identifica alimentación de 3,3 V, masa y señal. No se dispone de la identificación exacta del módulo ni de un relevamiento del orden de terminales; el funcionamiento observado no sustituye esa caracterización.

Interfaz con el accionamiento del láser. La señal de control del láser se genera mediante el canal 1 de TIM3 y se obtiene en PC6, disponible en el terminal 4 del conector CN10 de la NUCLEO-F446RE. Esta red se identifica como LASER_PWM.

La temporización del PWM se describe en la Sección 7. La red PC6 llega a la interfaz P-DA-01 y no directamente al diodo láser. El manual identifica una entrada PWM de 0/5–12 V, pero no especifica un umbral que garantice el reconocimiento de 3,3 V. No se dispone de una caracterización eléctrica de esta conexión; se registra como limitación en la Subsección 9.1.

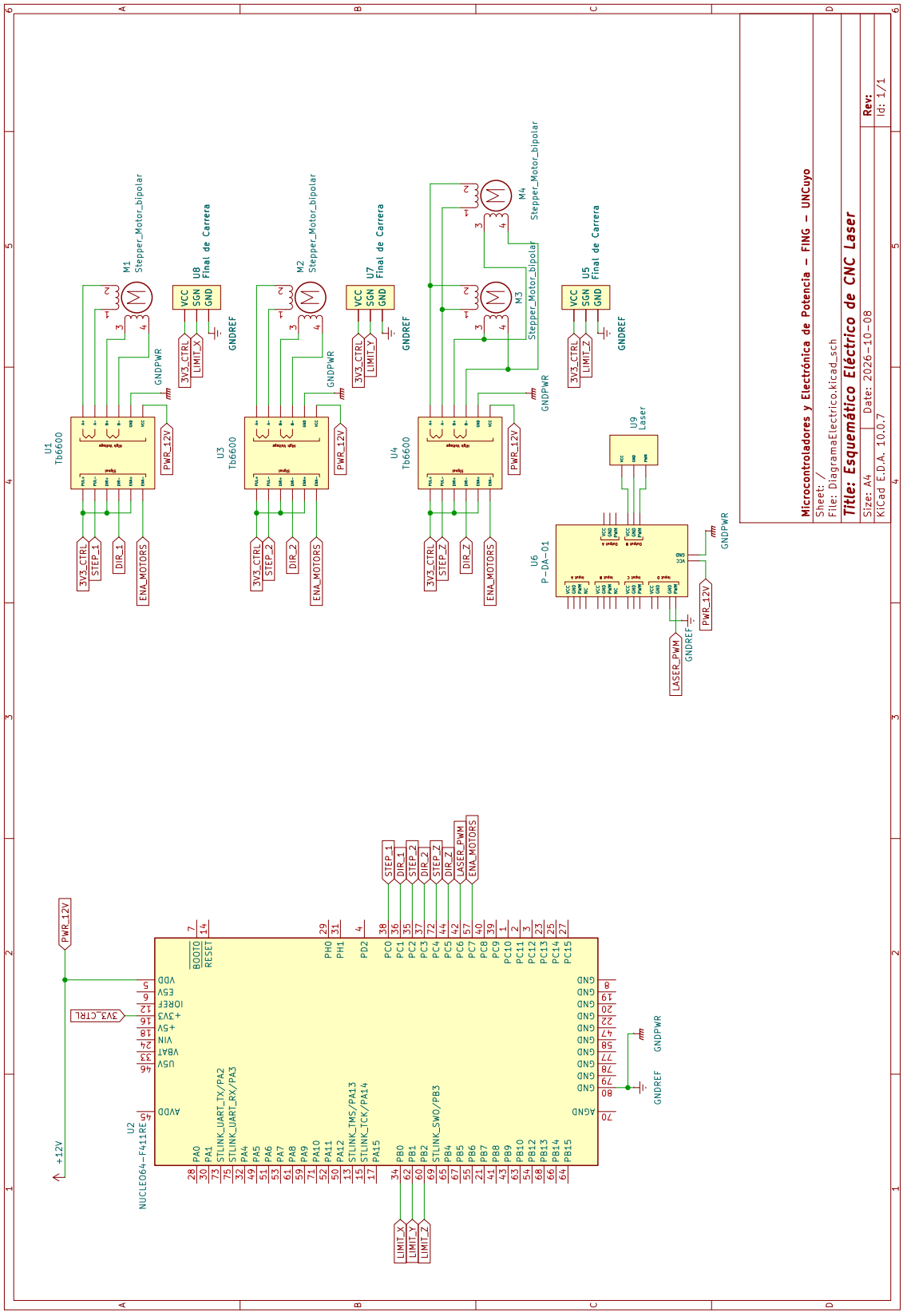


Figura 8. Esquema de interconexión elaborado en KiCad. El original utiliza un símbolo rotulado F411RE, mientras que la placa y el firmware corresponden a F446RE. La tabla de señales identifica las redes GPIO del proyecto; este esquema no acredita por sí solo la correspondencia física de todos los conectores.

4. Arquitectura del firmware

4.1. Organización modular

El firmware fue desarrollado en lenguaje C mediante una organización modular. Las responsabilidades del sistema se distribuyen entre módulos de interpretación de instrucciones, generación de movimiento, referenciación, control del láser y habilitación de los drivers.

Los módulos funcionales se organizan mediante archivos de encabezado, en los que se declara su interfaz pública, y archivos de implementación, en los que se definen las operaciones correspondientes. Los encabezados de configuración reúnen constantes y asignaciones de pines, sin requerir necesariamente un archivo de implementación.

El estado interno y las funciones auxiliares de los módulos se mantienen, cuando corresponde, mediante declaraciones con el especificador `static`. En las declaraciones a nivel de archivo, este especificador establece enlace interno y evita que sus identificadores sean utilizados directamente desde otras unidades de compilación. De esta manera, el acceso al módulo se realiza mediante las funciones declaradas en su encabezado.

Las variables locales declaradas con `static` conservan su valor entre llamadas sucesivas. Este recurso es utilizado, por ejemplo, para recordar las condiciones de continuidad entre segmentos de movimiento. El ocultamiento del estado interno permite modificar su representación sin alterar la interfaz utilizada por los demás módulos.

Las dependencias funcionales se muestran en la Figura 10. Cada bloque representa un módulo C con una interfaz propia. Las marcas de visibilidad son una adaptación gráfica y no implican una implementación orientada a objetos.

La Figura 9 muestra una selección de archivos de la carpeta `Code/`, utilizada tanto en la copia local como en el repositorio revisado. El contraste se realizó con la revisión `0ce7b21`, cuyo código fuente coincide con el utilizado en este informe [7].

```
Code/
|-- CNC_laser.ioc
|-- Core/
|   |-- Inc/
|   |   |-- main.h, corexy_config.h
|   |   |-- gcode_parser.h, homing.h
|   |   |-- stepper.h, z_axis.h
|   |   |-- laser.h, motor_enable.h, watchdog.h
|   |   `-- ... encabezados de soporte
|   |-- Src/
|   |   |-- main.c, gcode_parser.c
|   |   |-- stepper.c, z_axis.c, homing.c
|   |   |-- laser.c, motor_enable.c, watchdog.c
|   |   |-- stm32f4xx_it.c, stm32f4xx_hal_msp.c
|   |   `-- ... archivos de soporte
|   `-- Startup/
|-- Drivers/
|   |-- CMSIS/
|   `-- STM32F4xx_HAL_Driver/
|-- STM32F446RETX_FLASH.ld
`-- STM32F446RETX_RAM.ld
```

Figura 9. Estructura seleccionada del proyecto de firmware. Se agrupan nombres en algunas filas y se abrevian los archivos de soporte y las bibliotecas.

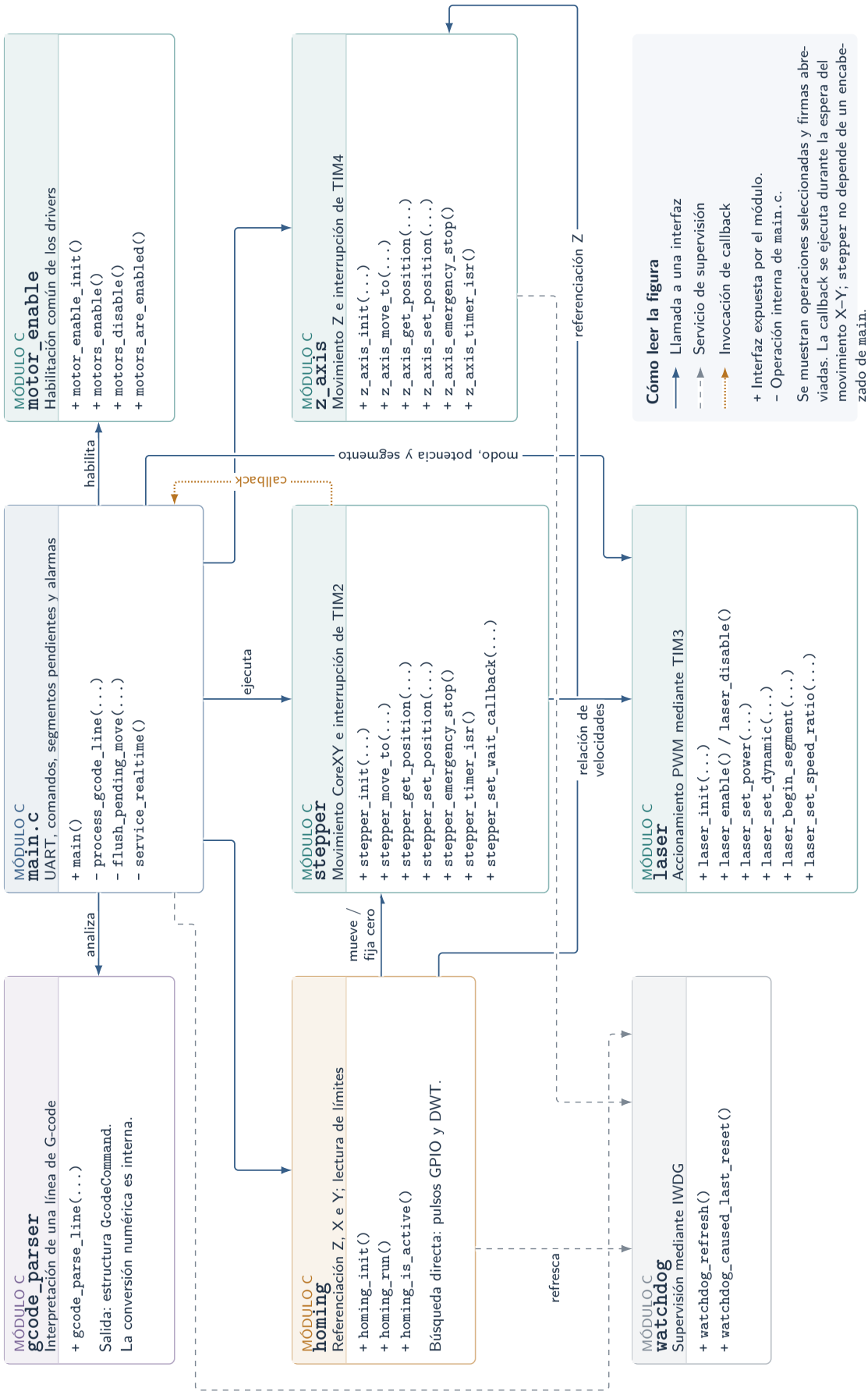


Figura 10. Organización modular del firmware. Cada bloque identifica un módulo C, su responsabilidad y una selección de sus operaciones. Las flechas indican llamadas a interfaces; se distinguen las dependencias de supervisión y la invocación mediante callback. Las firmas se presentan abreviadas.

Coordinación de la aplicación: `main.c`. En este archivo se inicializan los periféricos y los módulos funcionales, y se implementa el bucle principal. También se construyen las líneas recibidas, se atienden los comandos especiales y se determinan las acciones correspondientes a las instrucciones G-code. Se conservan el modo de coordenadas, el avance modal, el segmento X–Y pendiente y las banderas asociadas a la comunicación y a la alarma.

Interpretación de instrucciones: `gcode_parser`. Este módulo transforma una línea de texto en una estructura `GcodeCommand`. En ella se identifican las coordenadas, el avance, el parámetro de accionamiento del láser y los comandos reconocidos. El módulo realiza la interpretación de la instrucción; la decisión sobre su ejecución y la conservación del estado modal corresponden a la aplicación principal.

Movimiento CoreXY: `stepper`. Se implementa la transformación de las consignas X–Y en desplazamientos de los motores A y B, junto con la coordinación de los pasos y el perfil de velocidad. El estado del movimiento y las posiciones lógicas se conservan internamente. La generación de pulsos es atendida mediante interrupciones de TIM2, mientras que la interfaz permite solicitar desplazamientos, consultar o establecer la posición lógica y detener el movimiento.

Movimiento del eje Z: `z_axis`. Se implementa el accionamiento del eje Z mediante consignas de posición y avance. El módulo conserva la posición lógica y el estado del desplazamiento, y actualiza los pasos mediante interrupciones de TIM4. Su ejecución es independiente del mecanismo de almacenamiento y continuidad de segmentos X–Y.

Referenciación: `homing`. Se implementa la secuencia de búsqueda de los finales de carrera en el orden Z, X e Y. La maniobra comprende una aproximación inicial, un retroceso, una segunda aproximación a menor velocidad y una separación final del contacto. Se utilizan las funciones de movimiento de los módulos CoreXY y Z, y se mantiene una bandera que identifica la ejecución de la referenciación.

Accionamiento del láser: `laser`. Se administra la habilitación del láser y la actualización del ciclo de trabajo de la señal PWM generada por TIM3. El parámetro S se limita al intervalo configurado y se convierte en un valor de comparación del temporizador. Se distingue entre la deshabilitación del láser y la anulación de su salida durante un movimiento rápido. En modo M4, `stepper` actualiza la relación entre la velocidad comandada y la de crucero para ajustar el PWM; no se mide la velocidad física.

Habilitación de los drivers: `motor_enable`. Se administra la señal común de habilitación de los tres drivers y se conserva su estado lógico. La interfaz permite habilitar, deshabilitar y consultar dicho estado, y es utilizada por la aplicación para atender los comandos M17 y M84.

Supervisión: `watchdog`. El módulo consulta la causa del reinicio y refresca IWDG. Se utiliza desde el servicio principal, las esperas de Z y los retardos del homing. Un reinicio causado por watchdog activa una alarma al arrancar; no restaura la posición física anterior.

Configuración del movimiento: `corexy_config.h`. Este encabezado reúne las asignaciones de pines y los parámetros de conversión entre desplazamiento y pasos de los motores CoreXY, junto con constantes del perfil de velocidad y límites del área de trabajo. Los parámetros correspondientes al eje Z se encuentran definidos en `z_axis.h`.

4.2. Inicialización y programa principal

Al comenzar la ejecución se inicializa la biblioteca HAL, se configura el reloj del sistema y se establecen los parámetros de los GPIO, USART2 y los temporizadores TIM2, TIM3 y TIM4. También se inicializa el watchdog independiente IWDG.

Posteriormente se inicializan los módulos de referenciación, habilitación de motores, movimiento del eje Z, movimiento CoreXY y control del láser. También se inicia la recepción de datos por interrupción y se transmite el mensaje de bienvenida. Si el reinicio anterior fue provocado por el watchdog, se activa la alarma y se informa la pérdida de referencia de posición. La limpieza mediante `$X` no sustituye una nueva referenciación, aunque el programa no la exige antes de aceptar otra consigna.

Una vez completada la inicialización, el programa permanece en un lazo principal. En él se atienden los mensajes pendientes, se verifica la condición de ejecución del movimiento X–Y almacenado y se extraen los caracteres del buffer de recepción para construir líneas completas.

Cada línea recibida es procesada mediante la función `process_gcode_line()`. Los comandos especiales son atendidos directamente, mientras que las instrucciones G-code son analizadas por el módulo `gcode_parser`. La estructura obtenida es utilizada para determinar las acciones sobre el movimiento, el láser y la habilitación de los motores.

4.3. Coordinación de los módulos

`main.c` mantiene los modos y el segmento pendiente, y solicita la ejecución mediante `flush_pending_move()`. Esta función fija el estado del láser para el segmento y llama a `stepper_move_to()`. El tratamiento de instrucciones y la anticipación se desarrollan en la Sección 5; la conversión CoreXY y la temporización, en la Sección 6.

Las consignas Z se ejecutan mediante `z_axis_move_to()`. El módulo `homing` utiliza operaciones de CoreXY y Z para referenciar la máquina, mientras que `motor_enable` administra la señal común de los drivers.

4.4. Ejecución mediante interrupciones

Las interrupciones de USART2 reciben los caracteres; las de TIM2 y TIM4 ejecutan los pasos de CoreXY y Z. TIM3 genera el PWM mediante hardware. Las funciones de movimiento esperan a que termine el segmento, mientras las interrupciones habilitadas siguen atendiendo los periféricos.

Durante la espera X–Y, una callback registrada por `main.c` atiende mensajes y refresca el watchdog. La espera Z y los retardos de homing refrescan el watchdog, pero no invocan ese mismo servicio de mensajes. El procesamiento de nuevas líneas sigue siendo secuencial. Los buffers y la respuesta al programa de computadora se describen en la Sección 5.

Las variables compartidas que lo requieren se declaran `volatile`. Este calificador expresa accesos que pueden cambiar fuera del flujo observado por el compilador; no sustituye la sincronización ni convierte una secuencia de operaciones en atómica. Su función es distinta del enlace interno establecido por `static`.

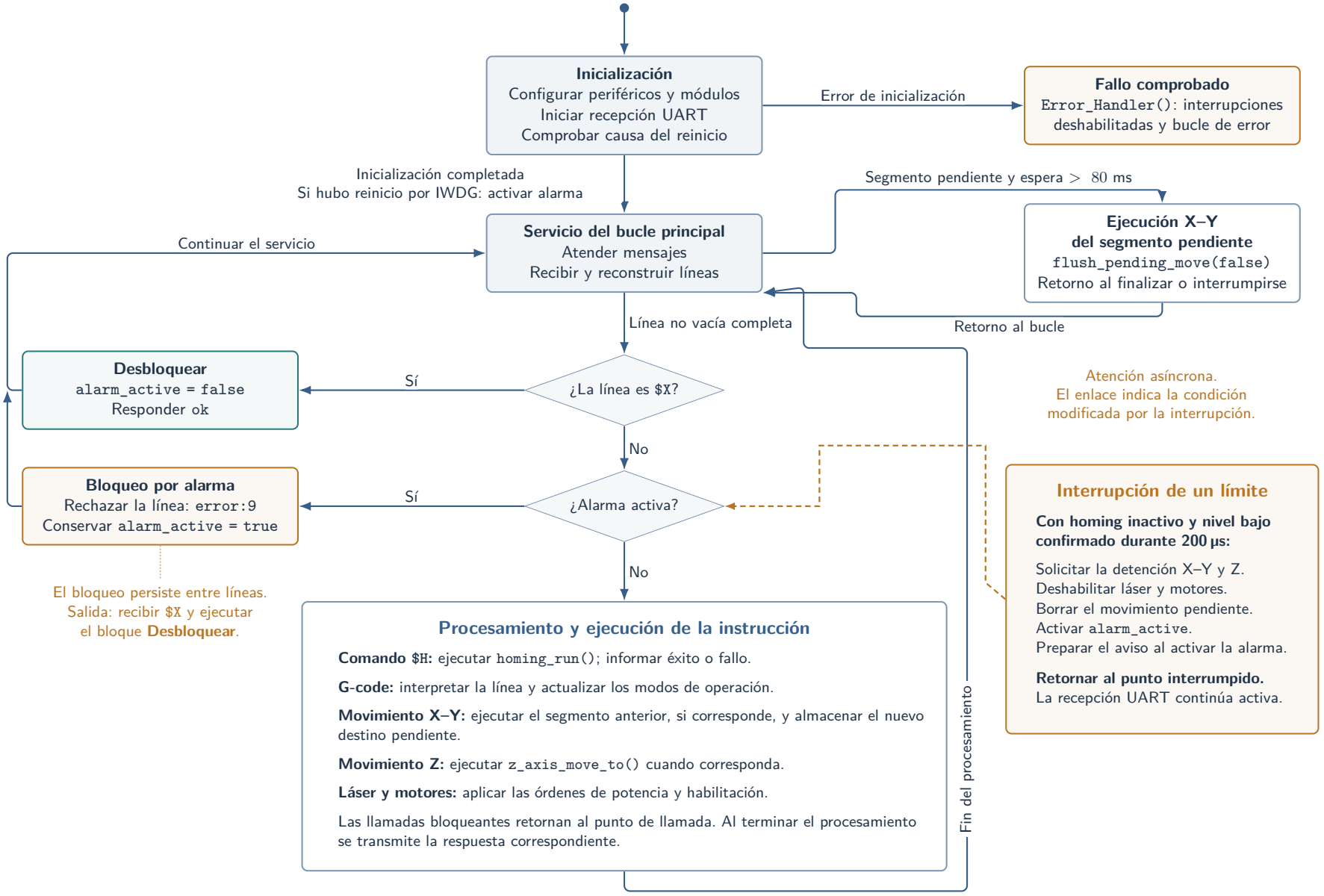
4.5. Representación de los estados de operación

El funcionamiento de la aplicación se representa mediante el diagrama unificado de la Figura 11. Se trata de un modelo funcional obtenido a partir del flujo de control y de las banderas del programa; no corresponde a una máquina de estados implementada mediante una única variable de estado.

El recorrido principal comprende la inicialización, el servicio del bucle y el procesamiento de las líneas recibidas. Antes de interpretar cada línea se reconoce el comando `$X` y se comprueba la bandera de alarma. Con alarma activa, las demás líneas son rechazadas. El bucle continúa atendiendo los mensajes de estado y bienvenida, así como la recepción UART. Las llamadas bloqueantes de movimiento y referenciación retornan al punto desde el que fueron invocadas.

La atención de un final de carrera es asíncrona. Fuera del *homing*, cuando la entrada permanece activa durante la comprobación de 200 µs, se solicita detener los movimientos, se deshabilitan el láser y los motores, se descarta el segmento pendiente y se activa la alarma. La línea discontinua del diagrama indica la modificación de esta condición; no representa un salto del programa principal hacia otro bucle de ejecución.

El retorno de la interrupción permite continuar el contexto interrumpido. Por ello, el bloqueo de nuevas líneas no equivale a cancelar todo el procesamiento de una instrucción ya iniciada. Además, la ejecución por tiempo de espera del segmento pendiente no comprueba por sí misma la bandera de alarma. Estas particularidades delimitan el alcance de la protección implementada y justifican revisar la inhibición del movimiento en todas las rutas de ejecución ante una alarma.



Lectura del diagrama. El bucle permanece activo aun con alarma: `$X` libera el bloqueo de nuevas líneas. Las solicitudes de estado y bienvenida se atienden por el servicio de mensajes. La interrupción no reinicia el flujo ni vuelve a comprobar una línea ya iniciada.

Figura 11. Diagrama funcional unificado del firmware. Las flechas continuas representan el flujo principal; el enlace discontinuo muestra el efecto de la interrupción de límite sobre la condición de alarma. Las operaciones agrupadas en el bloque de procesamiento se ejecutan según la instrucción recibida.

5. Comunicación e interpretación de G-code

5.1. Generación y envío de instrucciones mediante LaserGRBL

La preparación de los trabajos y el envío de instrucciones a la máquina se realizan mediante LaserGRBL. Este programa permite importar imágenes, establecer parámetros del trabajo y generar una secuencia de instrucciones G-code. También proporciona una interfaz para transmitirlas al controlador y recibir sus respuestas [4].

La utilización de una herramienta existente permite delimitar el desarrollo al sistema embebido encargado de interpretar las consignas y accionar la máquina. La implementación de los algoritmos de procesamiento de imágenes y generación de trayectorias no forma parte del alcance del proyecto. Sin embargo, las instrucciones generadas deben pertenecer al conjunto reconocido por el firmware y utilizar parámetros compatibles con la configuración de la CNC.

En los trabajos de grabado de imágenes pueden generarse secuencias de segmentos lineales con diferentes valores de accionamiento del láser. La ejecución de estas trayectorias requiere conservar el orden de las instrucciones y asociar cada segmento con su consigna de velocidad y de accionamiento. La coordinación mecánica del barrido se desarrolla en la sección de generación de movimiento.

5.2. Comunicación serie y tratamiento de mensajes

La comunicación con la computadora se establece mediante USART2, configurada a 115200 bit/s, con ocho bits de datos, un bit de parada y sin paridad. La conexión USB del ST-LINK permite acceder a esta interfaz mediante un puerto serie virtual.

La recepción se realiza por interrupción, de a un carácter. Los caracteres ordinarios son almacenados en un buffer circular y posteriormente extraídos por el programa principal. Al encontrarse un retorno de carro o un salto de línea, se termina la cadena y se procesa la instrucción construida. Las líneas vacías no son enviadas al parser.

El buffer circular dispone de 256 posiciones, de las cuales 255 pueden contener caracteres pendientes. El buffer de construcción de líneas dispone de 96 posiciones, incluyendo el terminador de cadena. Estos límites condicionan la cantidad de información que puede conservarse mientras el programa principal se encuentra ocupado. Si el buffer circular se llena, se descartan los caracteres nuevos; si una línea supera 95 caracteres, su parte restante no se conserva. No se genera un error específico por estas condiciones.

Para mantener el intercambio con LaserGRBL se utilizan respuestas y mensajes con formato compatible con su interfaz [8]: `ok`, `error:n`, `ALARM:n` y `[MSG:...]`. Las instrucciones que alcanzan el final del procesamiento reciben una respuesta `ok`; determinados errores producen una respuesta `error:n` y un retorno anticipado.

La respuesta `ok` no debe interpretarse como una confirmación de que todos los desplazamientos asociados ya finalizaron. En particular, una consigna X–Y puede haber quedado almacenada como movimiento pendiente cuando se transmite esta respuesta.

Los caracteres `?` y `0x18` son tratados por la rutina de recepción sin incorporarse a las líneas G-code. El primero solicita un reporte de estado; el segundo solicita el mensaje de bienvenida y el restablecimiento de los offsets de trabajo. Las acciones correspondientes se atienden mediante banderas y la función `service_realtime()`, desde el contexto del programa principal.

Durante los movimientos X–Y, esta función también es invocada desde la espera de la rutina de movimiento. De esta manera pueden atenderse solicitudes de estado sin comenzar el procesamiento de otra línea G-code.

5.3. Interpretación de las instrucciones

Las instrucciones recibidas son cadenas de caracteres, mientras que los módulos de control requieren valores numéricos y acciones identificadas. Por ello se utiliza un parser, es decir, una función que reconoce los elementos de la instrucción y los transforma en una representación que puede ser utilizada por el programa.

El análisis se implementa en `gcode_parser.c` mediante `gcode_parse_line()`. Su resultado se almacena en una estructura `GcodeCommand`, que contiene el tipo de movimiento, los parámetros numéricos y banderas que indican cuáles estaban presentes en la línea.

Cada palabra se identifica mediante una letra y un valor numérico. Por ejemplo, en `G1X12.5Y8F1200S300`, la letra G identifica el movimiento, X e Y indican coordenadas, F establece el avance y S representa la consigna de accionamiento del láser. Las banderas permiten distinguir un parámetro omitido de uno cuyo valor es explícitamente cero.

El recorrido de la cadena admite letras mayúsculas o minúsculas y palabras consecutivas sin espacios. Se descartan los comentarios iniciados por punto y coma y se omiten los comentarios entre paréntesis en las líneas procesadas. Las líneas que comienzan, después de los espacios iniciales, con un comentario son descartadas.

La Figura 12 distingue recepción, análisis y ejecución. La conversión numérica se realiza mediante `parse_gcode_number()`, que reconoce un signo opcional, dígitos y una parte decimal opcional. Esta función evita interpretar la secuencia `0X10` de una instrucción `G0X10` como un número hexadecimal. El comentario de implementación documenta que una conversión de propósito general podía consumir la X y su valor como parte del número, en lugar de reconocer una nueva palabra. El lector decimal corrige ese caso; no admite notación científica y no sustituye una validación sintáctica completa.

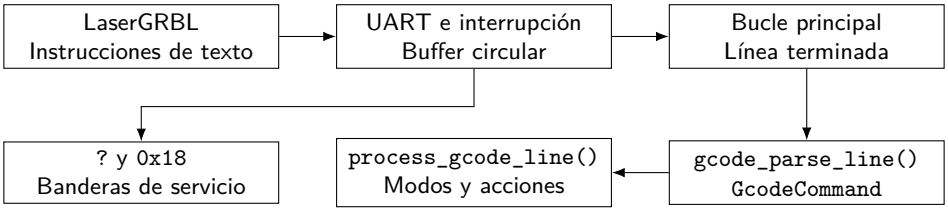


Figura 12. Recorrido de las instrucciones. Los comandos de sistema se resuelven en `main.c`; los caracteres de servicio no ingresan al parser de líneas.

El parser no ejecuta movimientos ni modifica directamente las salidas. La estructura obtenida es entregada a `process_gcode_line()`, en `main.c`, donde se aplica el estado modal, se calculan los destinos y se invocan las operaciones correspondientes.

5.4. Comandos reconocidos y estado modal

La implementación reconoce un subconjunto de instrucciones, seleccionado para las funciones requeridas por el proyecto. Este conjunto se resume en la Tabla 5.

Tabla 5. Instrucciones y parámetros reconocidos por el firmware.

Instrucción	Función
`G0`	Traslado con salida del láser anulada
`G1`	Movimiento lineal con avance programado
`G90`, `G91`	Coordenadas absolutas o relativas
`G92`, `G92.1`	Establecer o borrar offsets X–Y
`X`, `Y`, `Z`	Consignas de posición
`F`	Avance expresado en mm/min
`S`	Consigna de accionamiento del láser
`M3`, `M4`	Habilitación constante o dinámica
`M5`	Deshabilitación del láser
`M17`, `M84`	Habilitación o liberación de motores

Los comandos de sistema se atienden directamente en `main.c`. Entre ellos se encuentran `$H`, para solicitar la referenciación; `$X`, para limpiar la condición de alarma; `$J=`, para solicitar movimientos manuales; y `$$`, para consultar la configuración que el controlador informa al programa de computadora. LaserGRBL y GRBL son componentes distintos: el primero genera y envía instrucciones desde la computadora; en la NUCLEO se ejecuta el firmware propio [3].

El modo de coordenadas y el avance programado se conservan entre instrucciones. En los movimientos manuales introducidos mediante `$J=`, la selección de coordenadas absolutas o relativas se aplica a esa instrucción sin modificar el modo del programa.

Los offsets de trabajo permiten diferenciar las coordenadas utilizadas por el trabajo de las posiciones lógicas de máquina. En las consignas absolutas X–Y, los offsets se suman a las coordenadas recibidas para obtener el destino utilizado por el controlador.

Las instrucciones desconocidas detectadas por el parser se identifican mediante la bandera `has_unknown`. En ese caso se rechaza la línea completa y se transmite una respuesta de error. Este tratamiento no constituye una implementación

completa del lenguaje G-code ni una validación exhaustiva de todas sus combinaciones. Los códigos G y M se convierten parcialmente a enteros y algunas secuencias mal formadas pueden ser aceptadas. Los comandos `$H` y `$X` requieren, además, el formato literal previsto por `main.c`.

5.5. Anticipación de segmentos mediante lookahead

Para reducir las detenciones entre segmentos consecutivos se implementa una anticipación de un segmento, denominada *lookahead*. Una consigna X–Y válida es almacenada en una estructura `PendingMove`, que conserva el destino, el avance, la dirección normalizada y la información asociada al láser.

Cuando se recibe el siguiente destino X–Y válido, se compara su dirección con la del segmento pendiente. La comparación se realiza mediante el producto escalar de ambos vectores normalizados:

$$c = \hat{\mathbf{d}}_{\text{pendiente}} \cdot \hat{\mathbf{d}}_{\text{nuevo}}.$$

Si se cumple $c > 0{,}999$, los segmentos se consideran suficientemente alineados y con el mismo sentido. Esta condición es utilizada para solicitar que el segmento pendiente finalice a velocidad de crucero. En caso contrario, se solicita su desaceleración final.

El segmento pendiente es ejecutado mediante `flush_pending_move()`, y el nuevo segmento queda almacenado para la comparación posterior. Al calcular una nueva consigna relativa, se considera como referencia el destino del pendiente, de modo que las instrucciones mantienen su secuencia lógica.

Si no se recibe una continuación, el bucle principal ejecuta el pendiente al comprobar que transcurrieron más de 80 ms desde su almacenamiento. Esta condición es evaluada por software y no corresponde a una interrupción que se produzca exactamente al cumplirse dicho tiempo.

También se ejecuta el pendiente antes de determinadas operaciones que requieren conservar el orden, como un movimiento Z, M5, M84, un cambio de offsets o una actualización de S sin coordenadas X–Y.

La información del láser asociada al segmento es aplicada al comenzar su ejecución. De esta manera, la recepción de una consigna posterior no adelanta el valor S del nuevo segmento. El criterio de continuidad compara direcciones, pero no verifica igualdad de avances; un cambio de F entre segmentos alineados puede introducir una discontinuidad de velocidad.

Tabla 6. Ejemplo de orden de ejecución del lookahead; no representa una medición de tiempos.

Evento	Acción	Pendiente al finalizar
Llega segmento 1	Se almacena; todavía no se mueve	Segmento 1
Llega segmento 2	Se compara la dirección y se ejecuta 1	Segmento 2
Llega segmento 3	Se compara la dirección y se ejecuta 2	Segmento 3
Sin continuación	Timeout mayor que 80 ms: se ejecuta 3	Ninguno

La Tabla 6 muestra que el último segmento requiere una condición de vaciado. Su valor S se aplica al ejecutar ese segmento, no al recibir el siguiente.

Este mecanismo conserva un solo segmento pendiente y utiliza un criterio geométrico de continuidad. No constituye un planificador de múltiples bloques con optimización general de velocidades.

5.6. Alcance de la compatibilidad con LaserGRBL

El firmware implementa las funciones de comunicación y los comandos necesarios para el uso previsto con LaserGRBL. La consulta $$ informa los valores $30, $31 y $32, asociados al intervalo de S y a la identificación del modo láser.

La consigna máxima de S está configurada en 1000. Este valor debe ser coherente con la configuración utilizada al generar el trabajo. Las instrucciones M3 y M4 seleccionan accionamiento constante y dinámico, respectivamente; el comportamiento del PWM se desarrolla en la sección de control del láser.

La compatibilidad se limita a las funciones implementadas. La transmisión de un mensaje de bienvenida con formato GRBL no implica disponer de todas las instrucciones, configuraciones o mecanismos de planificación del firmware original. Por ello, el G-code utilizado debe verificarse respecto del conjunto admitido. Por ejemplo, G21, G2 y G3 no se reconocen en esta

versión. El código conserva `g_last_motion`, pero no lo utiliza para determinar si una línea sin G explícita es rápida; por ello, no se garantiza la herencia completa de G0.

El reporte de estado utiliza `Idle` o `Alarm`, conserva Z y los campos de avance y accionamiento en cero, e informa los offsets X–Y mediante `WCO`. No constituye una medición del estado físico ni un reporte completo de GRBL.

6. Generación y coordinación del movimiento

6.1. Cinemática y elección del mecanismo CoreXY

El desplazamiento del cabezal en el plano X–Y se realiza mediante un mecanismo CoreXY. En esta arquitectura, dos motores fijos a la estructura actúan sobre una transmisión de correas que combina sus movimientos para producir los desplazamientos cartesianos del cabezal. Por lo tanto, ninguno de los motores corresponde exclusivamente al eje X o al eje Y.

Criterios de elección. La adopción de CoreXY estuvo vinculada con la utilización del diseño mecánico de referencia, cuya estructura y disposición de transmisión fueron conservadas. Esta elección permitió concentrar el desarrollo del proyecto en las interfaces eléctricas, la interpretación de instrucciones y la implementación del control de movimiento.

Desde el punto de vista funcional, la arquitectura resulta adecuada para el posicionamiento de un cabezal láser. A diferencia de una disposición cartesiana de ejes independientes en la que un eje transporta el motor del otro, CoreXY permite mantener ambos motores de accionamiento fijos al bastidor. De esta manera, se evita incorporar su masa a los conjuntos móviles, lo que favorece la capacidad de aceleración del mecanismo [5].

Esta característica es de interés para el grabado, donde se ejecutan desplazamientos sucesivos y cambios de sentido. Además, el procesamiento mediante láser no requiere transmitir las fuerzas de contacto propias de una herramienta de mecanizado, por lo que una transmisión por correas constituye una alternativa compatible con la aplicación prevista.

CoreXY conserva un espacio de trabajo descrito mediante coordenadas cartesianas. La diferencia respecto de los ejes accionados de manera independiente se encuentra en la relación entre esas coordenadas y los motores: en CoreXY debe realizarse una transformación para obtener las consignas de ambos actuadores.

Principio de funcionamiento. El acoplamiento se obtiene mediante el recorrido de las correas y su vinculación con el conjunto móvil. Con la convención adoptada, las contribuciones de los motores se suman para producir el desplazamiento en X y se restan para producir el desplazamiento en Y [6].

Puede interpretarse que un movimiento del motor A, manteniendo fijo el motor B, produce una combinación de desplazamientos en X e Y. La actuación del motor B aporta otra combinación que permite reforzar una componente y cancelar la otra. Así, la actuación coordinada de ambos motores permite obtener movimientos sobre los ejes cartesianos y trayectorias intermedias.

Para expresar esta relación se definen Δa y Δb como los desplazamientos lineales equivalentes de la transmisión asociados a los motores A y B. Estas magnitudes se expresan en unidades de longitud; su conversión a cantidades de pulsos se realiza posteriormente mediante la relación de transmisión configurada.

La transformación cinemática directa se expresa como:

$$\Delta x = \frac{\Delta a + \Delta b}{2}, \qquad \Delta y = \frac{\Delta a - \Delta b}{2}. \tag{1}$$

En consecuencia, cuando ambos motores producen desplazamientos equivalentes de igual magnitud y signo, la componente en Y se anula y se obtiene un movimiento exclusivamente en X. Cuando sus desplazamientos tienen igual magnitud y signos opuestos, se anula la componente en X y se obtiene un movimiento exclusivamente en Y.

Transformación de las consignas. Las instrucciones G-code establecen destinos cartesianos, mientras que los actuadores requieren consignas de giro. Por ello, el firmware utiliza la transformación cinemática inversa para convertir cada desplazamiento solicitado en las actuaciones de los motores A y B:

$$\Delta a = \Delta x + \Delta y, \qquad \Delta b = \Delta x - \Delta y. \tag{2}$$

Estas relaciones también permiten interpretar los movimientos diagonales. Cuando $\Delta x = \Delta y$, el desplazamiento correspondiente al motor B es nulo. Cuando $\Delta x = -\Delta y$, es nula la actuación del motor A.

Los signos dependen de la disposición de las correas, de la identificación de los motores y de la conexión de sus bobinas. Por ello, la convención del programa debe comprobarse mediante movimientos de prueba sobre cada eje.

La transformación determina la cantidad de movimiento requerida en cada actuador, pero no establece por sí sola su evolución temporal. Para conservar una trayectoria rectilínea, el firmware debe distribuir los pulsos de ambos motores de manera coordinada, manteniendo la proporción correspondiente al segmento solicitado. Esta tarea es realizada por el algoritmo de generación de pasos descrito en los apartados siguientes.

Ventajas y limitaciones. La principal ventaja de la arquitectura para esta aplicación es la reducción de masa móvil obtenida al mantener los motores sobre el bastidor. A ello se suma una transformación cinemática lineal, basada en sumas y restas, cuya implementación requiere un procesamiento reducido.

Como contrapartida, la transmisión presenta un recorrido de correas y una disposición de poleas más complejos que los de dos ejes accionados de forma independiente. El montaje requiere controlar la alineación de los elementos, la escuadra del mecanismo y la tensión de las correas. Un ajuste inadecuado puede aumentar el rozamiento, favorecer deformaciones o afectar el desplazamiento del cabezal [10].

El acoplamiento también implica que un error de actuación de uno de los motores puede afectar simultáneamente a las dos coordenadas cartesianas. Por otra parte, la elasticidad de las correas y la rigidez del conjunto limitan la correspondencia entre el movimiento comandado y el obtenido.

Por estas razones, la elección de CoreXY se justifica por su adecuación al cabezal y a las trayectorias previstas, junto con la disponibilidad del diseño mecánico adoptado. No se considera una garantía de mayor exactitud: las prestaciones alcanzadas deben comprobarse mediante ensayos sobre la máquina construida.

6.2. Conversión de desplazamientos en pulsos

La generación del movimiento requiere expresar las consignas de los motores como cantidades enteras de pulsos. Para ello se utiliza un factor de conversión K_s, expresado en pulsos por milímetro, que depende del ángulo de paso del motor, la subdivisión de pasos seleccionada en el driver y las características de la transmisión.

Para una polea dentada, el factor se calcula mediante:

$$K_s = \frac{N_{\mathrm{rev}}\, m}{z\, p}, \tag{3}$$

donde N_{rev} es la cantidad de pasos completos por revolución, m es el factor de subdivisión de pasos, z es la cantidad de dientes de la polea y p es el paso de la correa.

En la configuración del firmware se consideran motores de $1{,}8^\circ$ por paso, equivalentes a 200 pasos completos por revolución, subdivisión de $1/16$ de paso y poleas de 20 dientes con correas de paso de $2\,\mathrm{mm}$. Con estos parámetros se obtiene:

$$K_s = \frac{200 \times 16}{20 \times 2} = 80\,\mathrm{pulsos/mm}. \tag{4}$$

Este valor corresponde a los parámetros configurados en el programa y debe ser contrastado con los componentes y los ajustes utilizados en el montaje.

A partir del desplazamiento solicitado, las cantidades de pulsos con signo se determinan como:

$$N_A = \mathrm{round}\left[K_s(\Delta x + \Delta y)\right], \qquad N_B = \mathrm{round}\left[K_s(\Delta x - \Delta y)\right]. \tag{5}$$

La operación de redondeo permite obtener valores enteros compatibles con el accionamiento discreto de los motores. El signo de cada resultado establece el sentido de giro mediante la señal DIR, mientras que su valor absoluto determina la cantidad de pulsos que debe generarse en STEP.

En el módulo `stepper`, estas operaciones se realizan a partir de la diferencia entre el destino solicitado y la posición lógica conservada por el firmware. La configuración permite, además, invertir el sentido de X en la transformación cinemática para adecuarlo al montaje.

6.3. Interpolación y sincronización de los motores

La transformación cinemática determina las cantidades totales de pulsos necesarias para alcanzar el destino. Sin embargo, estas cantidades no definen por sí solas la trayectoria seguida durante el desplazamiento. También debe establecerse cómo se distribuyen los pulsos de ambos motores a lo largo del segmento.

Si todos los pasos de un motor fueran ejecutados antes que los del otro, podría alcanzarse el mismo destino ideal mediante una trayectoria diferente de la recta solicitada. Por ello, los accionamientos deben coordinarse durante la ejecución.

En el proyecto se utiliza una interpolación discreta basada en el algoritmo de Bresenham. Se identifica como accionamiento principal al motor que requiere la mayor cantidad de pulsos:

$$N_{\text{mayor}} = \text{máx}(|N_A|, |N_B|), \qquad N_{\text{menor}} = \text{mín}(|N_A|, |N_B|). \tag{6}$$

En cada evento de temporización se genera un pulso para el motor principal. La generación del pulso del motor secundario se decide mediante un acumulador de error, que permite distribuir sus pasos proporcionalmente durante el recorrido.

El acumulador se inicializa con la mitad entera de N_{mayor}. En cada evento se resta N_{menor}; cuando el resultado es negativo, se genera también un pulso para el motor secundario y se suma N_{mayor} al acumulador. De esta manera se conserva la proporción requerida sin efectuar una división en cada decisión de paso.

Por ejemplo, para un desplazamiento de $\Delta x = 10\,\text{mm}$ y $\Delta y = 5\,\text{mm}$, con el factor configurado de $80\,\text{pulsos/mm}$, se obtiene:

$$N_A = 1200, \qquad N_B = 400. \tag{7}$$

El motor A constituye el accionamiento principal y recibe un pulso en cada evento. Los 400 pulsos del motor B se distribuyen durante el segmento, aproximadamente uno cada tres eventos. La relación entre ambos accionamientos permite aproximar la trayectoria rectilínea mediante movimientos discretos.

Para visualizar la distribución sin una secuencia de 1200 eventos, la Tabla 7 utiliza $N_{\text{mayor}} = 12$ y $N_{\text{menor}} = 4$, con la misma proporción 3:1 del ejemplo. El error inicial es 6; la condición es estrictamente negativa.

Tabla 7. Secuencia ilustrativa de Bresenham calculada con la regla del firmware. Un 1 indica un pulso.

Evento	1	2	3	4	5	6	7	8	9	10	11	12
Principal	1	1	1	1	1	1	1	1	1	1	1	1
Secundario	0	1	0	0	1	0	0	1	0	0	1	0

Cuando ambos motores requieren igual cantidad de pulsos, se acciona cada uno en todos los eventos. Cuando uno de ellos no requiere desplazamiento, solo se generan pulsos para el otro.

La coordinación se realiza desde una misma rutina de interrupción de TIM2. De esta manera, las decisiones de paso de ambos motores comparten una referencia temporal y un contador de avance del segmento.

6.4. Temporización y perfil de velocidad

El parámetro F de la instrucción G-code representa el avance a lo largo de la trayectoria, expresado en milímetros por minuto. Para su utilización en los cálculos de movimiento se convierte a milímetros por segundo:

$$v = \frac{F}{60}. \tag{8}$$

La frecuencia de pasos del accionamiento principal debe ajustarse según la orientación y la longitud del segmento. Una misma frecuencia de motor no produce necesariamente la misma velocidad cartesiana en todos los movimientos CoreXY.

La longitud del desplazamiento solicitado se obtiene mediante:

$$L = \sqrt{(\Delta x)^2 + (\Delta y)^2}. \tag{9}$$

Para una velocidad cartesiana nominal v, la frecuencia de eventos del motor principal resulta:

$$f_{\text{mayor}} = v\,\frac{N_{\text{mayor}}}{L}. \tag{10}$$

En el firmware esta conversión se implementa mediante un factor que relaciona los pulsos del motor principal con la longitud cartesiana. Así, el avance solicitado se interpreta respecto del recorrido del cabezal y no de uno de los accionamientos por separado.

TIM2 se configura con una base temporal nominal de $1\,\mu$s. Durante la ejecución, su período se actualiza para establecer el intervalo hasta el siguiente evento de paso. La rutina de interrupción genera los pulsos, actualiza el contador de avance y calcula la temporización siguiente.

Cuando no se solicita continuidad entre segmentos, se implementan fases de aceleración, velocidad de crucero y desaceleración. El lookahead puede omitir rampas de entrada o salida; no elimina necesariamente las discontinuidades al cambiar el avance entre segmentos. Durante las rampas, la velocidad de pasos se calcula a partir de una relación de la forma:

$$v_p^2 = 2a_p n, \tag{11}$$

donde v_p es la velocidad expresada en pulsos por segundo, a_p es la aceleración en pulsos por segundo cuadrado y n representa la cantidad de eventos recorridos durante la aceleración o restantes durante la desaceleración.

El período entre eventos se obtiene como el inverso de la velocidad de pasos. El mínimo configurado es de 50 μs para X–Y y de 80 μs para Z. Estos valores limitan la temporización solicitada; los intervalos físicos también incluyen la ejecución de las rutinas. El avance máximo admitido por el procesamiento de líneas es de 3000 mm/min: las consignas F superiores se recortan y se informa un aviso, sin activar una alarma.

La distancia de rampa X–Y configurada es de 3 mm. La aceleración se calcula para alcanzar la velocidad de crucero en esa distancia: no corresponde a una aceleración fija de máquina. Si se acorta una rampa para adaptarla al segmento, el firmware vuelve a calcular la aceleración sin aplicar un límite independiente.

Las extensiones de las rampas se ajustan cuando el segmento no dispone del recorrido previsto para ambas fases. Asimismo, las condiciones de entrada y salida a velocidad de crucero pueden ser modificadas por el mecanismo de *lookahead*, descrito en la Subsección 5.5.

6.5. Ejecución del barrido de grabado

En el grabado por barrido, la imagen es representada mediante una secuencia de recorridos lineales. Durante cada línea se ejecutan segmentos consecutivos, con consignas de accionamiento del láser asociadas a la información que debe reproducirse sobre el material.

Al finalizar una línea se realiza un desplazamiento transversal para alcanzar la siguiente. Según la configuración de generación de trayectorias, el recorrido puede alternar su sentido entre líneas o incluir un traslado para comenzar nuevamente desde el mismo lado.

Las consignas cartesianas de cada segmento se transforman mediante la cinemática CoreXY y sus pulsos se distribuyen mediante la interpolación descrita. Para las líneas paralelas a X, ambos motores reciben cantidades iguales de pulsos con el mismo signo; para el avance transversal sobre Y, las cantidades son iguales y sus signos opuestos.

El valor S puede variar entre segmentos de una misma línea. Por lo tanto, la reproducción del grabado requiere coordinar el desplazamiento con la consigna del láser, que puede corresponder a una salida nula o a diferentes ciclos de trabajo del PWM.

En el modo M4, la implementación utiliza además la relación entre la velocidad calculada por el perfil y la velocidad de crucero para ajustar el accionamiento del láser. Esta velocidad es determinada por el programa; no corresponde a una medición del movimiento físico.

6.6. Movimiento independiente del eje Z

El desplazamiento del eje Z se implementa en el módulo `z_axis` y se temporiza mediante TIM4. Su consigna se convierte en pulsos considerando los parámetros del motor, el micropaso y el avance del husillo.

Los dos motores de Z se encuentran documentados como conectados en paralelo a un mismo driver, por lo que reciben una única secuencia de accionamiento y no son controlados individualmente por el firmware.

El eje Z no participa de la interpolación CoreXY ni del almacenamiento de segmentos X–Y. Su ejecución se realiza de manera independiente para ajustar la posición vertical del cabezal. Por ello, la implementación no debe interpretarse como una interpolación simultánea de tres ejes.

6.7. Resolución y exactitud del posicionamiento

La conversión a cantidades enteras de pulsos introduce una discretización del movimiento. La subdivisión de pasos y la relación de transmisión determinan la resolución nominal de las consignas, pero no establecen por sí solas la exactitud del desplazamiento físico.

La coordinación de los pulsos permite conservar la relación de movimiento requerida por la cinemática CoreXY y aproximar la trayectoria programada. La exactitud resultante depende también de la correspondencia entre los parámetros configurados y el montaje real, de las holguras, de la elasticidad y tensión de las correas y de la ejecución de los pasos sin pérdidas.

El posicionamiento se realiza en lazo abierto. Las posiciones conservadas por el firmware son valores lógicos y no mediciones obtenidas mediante sensores de posición. En la implementación revisada, la posición lógica X–Y se actualiza al destino antes de finalizar el desplazamiento.

La exactitud y la repetibilidad deben evaluarse mediante ensayos de la máquina. El conteo lógico no comprueba el desplazamiento físico y, tras una detención, no debe confundirse con una posición medida.

7. Referenciación, límites y control del láser

7.1. Referenciación de los ejes

El posicionamiento se realiza en lazo abierto, a partir de los pasos comandados a los motores. Al encender la máquina, el valor inicial de las coordenadas no identifica por sí solo una ubicación física del cabezal. Para establecer esa correspondencia se implementa una rutina de referenciación, solicitada mediante el comando `$H`.

La función `homing_run()` ejecuta la búsqueda de los finales de carrera en el orden Z, X e Y. La referenciación de Z se realiza primero para llevar el cabezal hacia la referencia superior antes de los desplazamientos del plano. Los ejes se atienden secuencialmente; si una búsqueda falla, no se continúa con los restantes.

Para cada eje se realiza una primera aproximación al sensor, un retroceso de 8 mm, una segunda aproximación más lenta y una separación final de 4 mm. Los períodos nominales entre pulsos de búsqueda son de 200 μs y 2000 μs, respectivamente. La segunda aproximación reduce la velocidad con la que se alcanza el contacto, para disminuir los efectos dinámicos durante su accionamiento. La repetibilidad obtenida debe determinarse mediante ensayos.

La Tabla 8 resume las fases programadas para un eje. Los valores corresponden al firmware y no a una medición del ensayo.

Tabla 8. Secuencia programada de referenciación por eje.

Fase	Consigna o criterio de finalización	Finalidad
Primera aproximación	Pulsos con período nominal de 200 μs hasta contacto o máximo de iteraciones	Localizar el sensor.
Retroceso	Desplazamiento positivo de 8 mm	Separarse del punto de contacto.
Segunda aproximación	Pulsos con período nominal de 2000 μs hasta contacto o máximo de iteraciones	Reaproximarse a menor velocidad.
Separación final	Desplazamiento positivo de 4 mm	Dejar el contacto liberado y fijar el cero lógico.

El cero lógico se establece después de la separación final. Por lo tanto, la referencia de coordenadas no coincide con el punto de accionamiento del sensor, sino con la posición alcanzada tras ese retroceso. Si el ciclo completo finaliza correctamente, `main.c` fija las coordenadas lógicas en cero, borra los desplazamientos del origen de trabajo X–Y y transmite la respuesta `ok`.

Las búsquedas tienen un máximo de 40000 iteraciones en X e Y y de 80000 en Z, para evitar una ejecución indefinida si no se detecta el contacto. Estos valores son límites de la rutina y no representan los recorridos útiles de la máquina. Si la

referenciación falla, se informa `ALARM:9`. En la versión revisada, este mensaje no activa por sí solo la bandera `alarm_active`; por ello, no debe confundirse con el bloqueo establecido ante un límite durante la operación normal.

Durante las búsquedas se generan pulsos directamente mediante GPIO y se utiliza el contador de ciclos DWT para temporizar las esperas. Los retrocesos se solicitan a los módulos de movimiento CoreXY y Z. La rutina es bloqueante para el programa principal y refresca el watchdog durante sus retardos.

7.2. Finales de carrera y bloqueo por alarma

Se dispone de una entrada de final de carrera por eje. La lógica del firmware considera activo el nivel bajo. Durante la referenciación, los sensores son consultados por la rutina de búsqueda y su accionamiento se considera parte de la maniobra esperada.

Fuera del homing, las entradas son atendidas mediante interrupciones externas. Antes de aceptar un evento, la función `limit_still_low()` comprueba que la señal permanezca en nivel bajo durante una ventana de 200 μs. Si vuelve al nivel alto dentro de ese intervalo, el evento es descartado. Esta confirmación temporal permite rechazar pulsos breves, pero no sustituye el acondicionamiento eléctrico de la entrada.

Al confirmarse un límite, la callback solicita detener los movimientos CoreXY y Z, deshabilita el láser, invalida el segmento pendiente y deshabilita los drivers. También activa `alarm_active` y solicita el envío de `ALARM:1`, junto con la identificación del eje.

La condición de alarma es comprobada al comenzar el procesamiento de cada nueva línea. Mientras permanece activa, las líneas distintas de `$X` son rechazadas. El comando `$X` limpia la bandera, pero no habilita los motores ni reconstruye la referencia física. La habilitación se solicita mediante `M17`; si la posición dejó de ser confiable, debe realizarse una nueva referenciación antes de retomar el trabajo.

La comprobación al inicio de una línea no garantiza la cancelación de todo procesamiento que ya estuviera en curso cuando ocurrió la interrupción. En particular, una línea ya iniciada puede volver a almacenar un segmento, y la condición de ejecución por timeout no comprueba la alarma. El vaciado de ese segmento también habilita los motores. La revisión de estas condiciones constituye un punto de validación del manejo de alarmas. Los límites implementados son mecanismos de protección funcional del firmware y no constituyen una parada de emergencia independiente del microcontrolador.

7.3. Límites del espacio de trabajo

Además de los sensores físicos, el programa comprueba los destinos solicitados mediante límites de software. Los intervalos configurados son:

$$0 \le x \le 240 \text{ mm}, \qquad 0 \le y \le 200 \text{ mm}, \qquad 0 \le z \le 130 \text{ mm}. \tag{12}$$

Para X–Y, la comprobación se realiza sobre el destino expresado en coordenadas de máquina, después de aplicar el modo absoluto o relativo y los desplazamientos del origen de trabajo. En Z se verifica el destino calculado para ese eje. Si queda fuera del intervalo, se informa la condición y no se ejecuta ese desplazamiento; ello no activa la alarma de final de carrera.

Estos valores corresponden a la configuración del firmware y deben contrastarse con los recorridos disponibles en el montaje. Su correspondencia con la ubicación física depende de la referenciación y de la conservación de la posición. La implementación no exige haber completado un homing antes de aceptar los movimientos, por lo que dicha condición debe ser respetada en la operación.

7.4. Accionamiento y modulación del láser

El módulo `laser` administra la habilitación y el ciclo de trabajo de la salida PWM. El temporizador TIM3 genera la señal de 1 kHz por el canal 1, asociado al pin PC6. La interfaz eléctrica del cabezal y sus características se describen en la sección de hardware.

El parámetro `S` establece una consigna entre 0 y 1000. El firmware limita el valor a ese intervalo y lo convierte en un valor de comparación entre 0 y 999 para el temporizador. Esta consigna determina el accionamiento eléctrico, pero no constituye una medición de potencia óptica.

Modo de accionamiento constante. Con `M3`, el ciclo de trabajo programado se mantiene sin escalarlo con la velocidad. La habilitación no implica que deba emitirse inmediatamente: la aplicación efectiva de la salida depende de la consigna y del estado del segmento. En este modo, la detención del movimiento no fuerza por sí sola la salida PWM a cero.

Modo de accionamiento dinámico. Con `M4`, el ciclo de trabajo programado se escala mediante la relación entre la velocidad de consigna instantánea y la velocidad de crucero del segmento:

$$D_{\text{salida}} \approx D_{\text{programado}} \, \text{sat}_{[0,1]} \left(\frac{v_{\text{consigna}}}{v_{\text{crucero}}} \right). \tag{13}$$

La expresión representa el escalado aplicado, sujeto a la cuantización del temporizador. El módulo `stepper` actualiza esa relación durante la ejecución del movimiento CoreXY. El objetivo es reducir el accionamiento durante las rampas de velocidad y limitar la concentración de energía por unidad de recorrido. No se utiliza una medición de velocidad ni una realimentación de potencia óptica; la uniformidad alcanzada debe evaluarse sobre las piezas procesadas.

Traslados y deshabilitación. En los segmentos identificados como `G0`, se anula la salida independientemente de la consigna programada. La función `laser_begin_segment()` aplica el estado de traslado y el valor `S` asociados al segmento antes de su ejecución.

El comando `M5` deshabilita el láser y borra la consigna almacenada. En consecuencia, para volver a obtener una salida distinta de cero debe establecerse nuevamente una consigna `S`, además de habilitar el modo correspondiente. La inicialización y la rutina de atención de límites también llevan la salida a cero.

Estas acciones no deben extenderse a situaciones que el código no contempla expresamente: en particular, la recepción de `$H` no incorpora un apagado explícito del láser. Su inhibición durante toda la referenciación constituye una mejora a considerar en la lógica de habilitación.

8. Ensayos y resultados

8.1. Alcance de las pruebas

Los resultados de esta sección corresponden a las observaciones de puesta en marcha comunicadas por el equipo del proyecto. Se distinguen de los ejemplos numéricos y de la revisión del código de las secciones anteriores. La verificación se realizó mediante pruebas funcionales y observación directa del funcionamiento. Fueron comprobadas la rutina de referenciación, la respuesta ante los finales de carrera y la ejecución de un grabado sobre MDF.

Los resultados disponibles son cualitativos. No se dispone de registros de medición que permitan cuantificar la exactitud del posicionamiento, la repetibilidad de la referenciación o los tiempos de respuesta. Por lo tanto, las observaciones presentadas permiten describir el funcionamiento alcanzado, pero no establecer tolerancias ni prestaciones metrológicas de la máquina.

8.2. Referenciación y respuesta a límites

Durante las pruebas, la rutina de referenciación completó la secuencia de los tres ejes y permitió establecer la posición inicial de trabajo. Se reportaron rebotes durante el accionamiento de los finales de carrera. Este comportamiento no fue caracterizado mediante registros eléctricos ni mediciones de repetibilidad, por lo que su efecto sobre la ubicación del cero no pudo cuantificarse.

La observación identifica un punto a mejorar en el conjunto formado por el sensor, su montaje, el cableado y el tratamiento de la señal. Sin mediciones adicionales no puede atribuirse exclusivamente a una de estas causas.

También se comprobó la respuesta ante el accionamiento de los finales de carrera durante un movimiento. Se observó la detención de la máquina al alcanzar el límite. No se dispone de mediciones de latencia ni de distancia de detención, por lo que el resultado se limita a la comprobación funcional de esa respuesta.

Esta función no permite detectar obstáculos intermedios dentro del espacio de trabajo. El firmware no utiliza medición de corriente ni realimentación de posición para reconocer una colisión o una pérdida de pasos. Las posibles protecciones eléctricas internas de los drivers no equivalen a una detección de obstáculos por parte del sistema de control.

8.3. Prueba de grabado

Se realizó un grabado sobre una pieza de MDF de 5 mm de espesor nominal. La obtención del grabado permitió comprobar de manera integrada la ejecución de una trayectoria y el accionamiento del láser sobre el material.

No se dispone de un registro completo de los parámetros empleados ni de mediciones dimensionales del resultado. En consecuencia, esta prueba documenta la capacidad de producir un grabado, pero no permite establecer una relación cuantitativa entre velocidad, consigna de láser y calidad obtenida. Tampoco permite deducir la capacidad de corte a partir del espesor de la pieza utilizada.

El corte no se considera validado en esta etapa. Su comprobación queda condicionada a disponer de una superficie de apoyo adecuada para recibir la radiación que atraviese el material, sin deteriorar los elementos situados debajo. Por lo tanto, no se declara un espesor máximo de corte ni una cantidad de pasadas requerida.

8.4. Síntesis de los resultados

La Tabla 9 resume las comprobaciones disponibles y el alcance de sus conclusiones.

Tabla 9. Resultados cualitativos de las pruebas del prototipo.

Función	Resultado observado	Alcance y limitaciones
Referenciación	Se completó la secuencia de los tres ejes.	Se reportaron rebotes; no se cuantificó la repetibilidad del cero.
Finales de carrera	Se observó la detención durante un movimiento.	No se midieron el tiempo ni la distancia de detención.
Grabado	Se obtuvo un grabado sobre MDF de 5 mm.	No se dispone de una caracterización dimensional ni de los parámetros completos de procesamiento.
Corte	Sin validación documentada.	Se requiere una superficie de apoyo adecuada antes de comprobar esta prestación.

La Tabla 10 vincula los objetivos con el desarrollo y con la evidencia disponible. La presencia de una función en el código no se toma como resultado experimental de todas sus condiciones de operación.

Tabla 10. Relación entre objetivos, implementación y verificación disponible.

Objetivo	Implementación	Evidencia y alcance
Interpretar instrucciones	UART y `gcode_parser`	Funcionamiento integrado de grabado; no hay registro de cobertura de todas las variantes de comandos.
Coordinar X–Y	CoreXY, interpolación y TIM2	Trayectoria de grabado ejecutada; sin medición del error de seguimiento.
Posicionar Z	Generación de pasos mediante TIM4	Referenciación Z completada; el ajuste de enfoque no fue caracterizado por separado.
Establecer una referencia	Homing con tres sensores	Secuencia completada con rebotes reportados; sin medida de repetibilidad.
Comandar el láser	PWM y modos M3/M4	Grabado obtenido; sin comparación experimental entre modos ni medida de potencia óptica.

Las pruebas permiten documentar el funcionamiento de la referenciación, la respuesta a los límites y la integración de movimiento y láser para grabado. La evaluación de exactitud, repetibilidad y capacidad de corte requiere ensayos específicos adicionales.

9. Limitaciones y conclusiones

9.1. Limitaciones y alcance de la validación

El alcance demostrado corresponde a la referenciación, la respuesta funcional ante límites y el grabado documentados en la Sección 8. No se dispone de mediciones de exactitud, repetibilidad, latencia de detención o potencia óptica. El corte no fue validado sobre una superficie de apoyo adecuada.

La revisión del código identifica restricciones adicionales: la compatibilidad con GRBL es parcial, el lookahead conserva un segmento y no planifica transiciones generales de velocidad, y las posiciones informadas son lógicas. Las comprobaciones de alarma no cancelan todos los caminos de ejecución ya iniciados; el fallo de homing no enclava la alarma y no existe una inhibición explícita del láser al recibir `$H`. Estas restricciones de diseño se distinguen de los rebotes observados durante la puesta en marcha.

La documentación física también tiene un alcance limitado. La Tabla 11 identifica los datos que no pueden reconstruirse a partir del programa. Su ausencia no se sustituye por valores de catálogo de componentes similares.

Tabla 11. Alcance de la caracterización física disponible.

Elemento	Información disponible	Límite de la documentación
Bastidor y recorridos	Diseño de referencia y geometría general	No se dispone del relevamiento dimensional del montaje.
Motores y drivers	Modelo 17HS3401, lote e hipótesis de conversión del firmware	Sin ficha específica de corriente y par ni registro de ajustes DIP de la unidad.
Interfaces de control	GPIO de 3,3 V; datos comerciales de entradas	No se verificaron umbrales y corrientes de entrada ni márgenes de compatibilidad.
Esquema KiCad	Redes y señales identificadas	El símbolo representa F411RE; el proyecto utiliza F446RE. Se interpreta por las redes GPIO declaradas, no como pinout validado.
Prueba de grabado	Resultado sobre MDF de 5 mm	Sin registro completo de avance, consigna S y pasadas ni fotografía de la pieza incorporada al informe.

9.2. Mejoras previstas

Las observaciones de puesta en marcha y la revisión del diseño permiten definir los siguientes pasos. Se priorizó la integración funcional del prototipo; las modificaciones propuestas requieren diseño y verificación antes de ser incorporadas.

Revisión de los bloqueos del firmware. Se propone comprobar la alarma antes de iniciar cualquier movimiento y después de cada llamada bloqueante, descartar consignas pendientes al fallar la referenciación y mantener una condición explícita de posición válida. También se requiere inhibir el láser durante todo el homing. Estas modificaciones deben acompañarse con pruebas de las secuencias interrumpidas; no se consideran implementadas en la versión documentada.

Separación entre potencia y control. Se prevé verificar la barrera de aislamiento de las entradas de los TB6600 y comprobar que las conexiones de alimentación y masa no la anulen. La revisión debe incluir los niveles lógicos y las corrientes de entrada. Según los resultados, podrá incorporarse una interfaz de acondicionamiento o aislamiento adicional. La protección debe evaluarse sobre el circuito completo.

Cableado y compatibilidad electromagnética. Se propone separar los recorridos de potencia y señal, mantener próximo cada conductor a su retorno y utilizar pares trenzados y pantallas donde corresponda. En los motores, cada par debe reunir los conductores de una misma fase. Las terminaciones de las pantallas deben definirse en conjunto con el chasis, las referencias de señal y la tierra de protección.

Placa de interfaz y entradas de sensores. Una PCB específica permitiría concentrar conectores, protecciones y acondicionamiento, conservando inicialmente la NUCLEO. Para los finales de carrera se plantea incorporar resistencias de polarización externas y filtros RC compatibles con la lógica de contacto. Los valores deben atenuar perturbaciones sin introducir un retardo excesivo ni flancos incompatibles con las entradas digitales. Este criterio no se traslada directamente a las señales STEP y PWM.

Guiado, transmisión y alimentación. Se propone revisar la rectitud de las guías, las holguras, la alineación de los husillos Z y la tensión de correas, evaluando sus efectos mediante mediciones de posición. En la alimentación se prioriza una fuente comercial con capacidad y protecciones documentadas contra sobrecorriente, cortocircuito y sobretensión. Una entrada

universal de 100–240 V y 50/60 Hz ampliaría la compatibilidad con redes eléctricas, siempre que esté especificada por el fabricante.

Diagnóstico y caracterización. Se plantea ampliar las consultas por terminal para visualizar estados, alarmas, escalas de conversión y límites configurados. Una HMI podría presentar esa información al operario. La lectura de tensiones reales requeriría circuitos de medición; la modificación de parámetros requeriría comprobación de rangos y almacenamiento persistente. Los estados calculados deben distinguirse de las magnitudes medidas.

El siguiente ciclo de validación debe incluir mediciones de desplazamiento, repetición del homing y registro de las condiciones de grabado y corte. La identificación de estas mejoras expresa una revisión del diseño: su incorporación se difiere considerando el alcance del prototipo y el trabajo necesario para integrarlas y comprobar su efecto.

9.3. Conclusiones

Se implementó un sistema de control capaz de interpretar instrucciones G-code, generar movimientos coordinados del mecanismo CoreXY, accionar el eje Z y comandar el láser mediante PWM. La organización modular permitió distribuir la interpretación, la referenciación y la generación de pasos entre interfaces específicas, con coordinación desde `main.c`.

Las pruebas de puesta en marcha comprobaron la referenciación de los tres ejes, la detención al alcanzar los finales de carrera y la ejecución de un grabado sobre MDF. Estos resultados respaldan el funcionamiento integrado previsto por el objetivo general. La validación es cualitativa y no permite asignar tolerancias de posicionamiento, repetibilidad ni prestaciones de corte.

El trabajo integra contenidos de sistemas embebidos, temporización por interrupciones, comunicación serie y accionamiento de motores paso a paso. La principal limitación del posicionamiento es la ausencia de realimentación: el conteo de pulsos representa una posición lógica y no comprueba la ejecución física. Las observaciones de rebote y las restricciones del manejo de alarmas muestran que el funcionamiento demostrado debe distinguirse de una validación integral de confiabilidad y protección.

El objetivo de control y grabado se considera alcanzado al nivel de las comprobaciones documentadas. La capacidad de corte, la caracterización cuantitativa y la comprobación eléctrica de las interfaces delimitan el alcance de las conclusiones y constituyen la base para una siguiente revisión del prototipo.

Referencias

[1] Javierarnedo. Core xy: diseño estructural de referencia. Thingiverse, objeto 2818865. https://www.thingiverse.com/thing:2818865.

[2] LASER TREE. *LT-40W-F23: Laser Module*. Manual técnico aportado para el proyecto, secciones 1.1, 2.1 y 2.2. Copia incluida en `attachments/LT-40W-F23-manual.pdf`; sin fecha de edición indicada.

[3] LaserGRBL. Configuration. https://lasergrbl.com/configuration/.

[4] LaserGRBL. Raster image import. https://lasergrbl.com/usage/raster-image-import/.

[5] Ilan E. Moyer. CoreXY: Cartesian motion platform. https://corexy.com/.

[6] Ilan E. Moyer. CoreXY: Principle of operation. https://www.corexy.com/theory.html.

[7] Nicoop. CNC_Laser_micro: código del firmware. Repositorio GitHub. https://github.com/Nicoop/CNC_Laser_micro/tree/0ce7b2114290752d20ff9d09d34ad79a6539d2ce.

[8] Proyecto Grbl. Grbl v1.1 interface. https://github.com/gnea/grbl/wiki/Grbl-v1.1-Interface.

[9] STMicroelectronics. UM1724: STM32 Nucleo-64 boards (MB1136). Manual de usuario. https://www.st.com/resource/en/user_manual/DM00105823-.pdf.

[10] Voron Design. Secondary printer tuning: Gantry racking and belt tension. https://docs.vorondesign.com/tuning/secondary_printer_tuning.html.